

Index

Sr. No		Topic	Page No.	Signature
1		Practical 1 Write the following programs for Blockchain in Python :	3	
	a.	A simple client class that generates the private and public keys by using the built in Python RSA algorithm and test it		
	b.	A transaction class to send and receive money and test it.		
2		Practical 2 Write the following programs for Blockchain in Python :	7	
	a.	Create multiple transactions and display them.		
	b.	Create a blockchain, a genesis block and execute it.		
3		Practical 3 Write the following programs for Blockchain in Python :	13	
	a.	Create a mining function and test it.		
	b.	Add blocks to the miner and dump the blockchain.		
4		Practical 4 Implement and demonstrate the use of the following in Solidity:	20	
	a.	Variable		
	b.	Operators		
	c.	Loops		
	d.	Decision Making		
	e.	Strings		
5		Practical 5 Implement and demonstrate the use of the following in Solidity :	36	
	a.	Arrays		
	b.	Enums		
	c.	Structs		
	d.	Mappings		
	e.	Coversations		
	f.	Ether Units		
	g.	Special Variables		

Index

6		Practical 6 Implement and demonstrate the use of the following in Solidity :	44	
	a.	Functions		
	b.	View Functions		
	c.	Pure Functions		
	d.	Fallback Functions		
	e.	Function Overloading		
	f.	.Mathematical Functions		
	g.	Cryptographic Functions		
7		Practical 7 Implement and demonstrate the use of the following in Solidity :	55	
	a.	Contracts		
	b.	Inheritance		
	c.	Constructors		
	d.	Abstract Class		
	e.	Interfaces		
8		Practical 8 Implement and demonstrate the use of the following in Solidity :	61	
	a.	Libraries		
	b.	Assembly		
	c.	Events		
	d.	Error Handling		

Practical 1

Write the following programs for Blockchain in Python

- a) A simple client class that generates the private and public keys by using the built in Python RSA algorithm and test it
- b) A transaction class to send and receive money and test it.

→

```
# import libraries
import hashlib
import random
import string
import json import
binascii import
numpy as np import
pandas as pd import
pylab as pl import
logging import
datetime import
collections
pip install pycryptodome
```

```
# following imports are required by PKI
import Crypto import
Crypto.Random from Crypto.Hash
import SHA from Crypto.PublicKey
import RSA
from Crypto.Signature import PKCS1_v1_5
```

```
import binascii class
```

```
Client:
```

```
def __init__(self):
```

```
    random = Crypto.Random.new().read
```

```
self._private_key = RSA.generate(1024, random)
```

```
self._public_key = self._private_key.publickey()
```

```
self._signer = PKCS1_v1_5.new(self._private_key)
```

```
    @property def
```

```
identity(self):
```

```
    return binascii.hexlify(self._public_key.exportKey(format='DER')).decode('ascii')
```

```

class Transaction:
    def __init__(self, sender, recipient, value):
        self.sender = sender
        self.recipient = recipient
        self.value = value
        self.time = datetime.datetime.now()
    def to_dict(self):
        if self.sender == "Genesis":
            identity = "Genesis"
        else:
            identity = self.sender.identity
        return collections.OrderedDict({
            'sender': identity,
            'recipient': self.recipient,
            'value': self.value,
            'time': self.time})
    def sign_transaction(self):
        private_key = self.sender._private_key
        signer = PKCS1_v1_5.new(private_key)
        h = SHA.new(str(self.to_dict()).encode('utf8'))
        return binascii.hexlify(signer.sign(h)).decode('ascii')

Dinesh = Client()
Ramesh = Client()

t = Transaction(Dinesh, Ramesh.identity, 5.0)
signature = t.sign_transaction()
print(signature)

```

Output:



+ Code + Text



Editing



```
[2] # import libraries
import hashlib
import random
import string
import json
import binascii
import numpy as np
import pandas as pd
import pylab as pl
import logging
import datetime
import collections
```

```
[3] pip install pycryptodome
```

Collecting pycryptodome

Downloading https://files.pythonhosted.org/packages/ad/16/9627ab0493894a11c68e4600dbcc82f578c8ff06bc2980dcd016aaa9bd3/pycryptodome-3.10.1-cp35-abi3-manylinux2014_x86_64.whl 1.9MB 4.8MB/s

Installing collected packages: pycryptodome

Successfully installed pycryptodome-3.10.1



+ Code + Text



Editing



```
[5] import Crypto
import Crypto.Random
from Crypto.Hash import SHA
from Crypto.PublicKey import RSA
from Crypto.Signature import PKCS1_v1_5

import binascii
```

```
import binascii
class Client:
    def __init__(self):
        random = Crypto.Random.new().read
        self._private_key = RSA.generate(1024, random)
        self._public_key = self._private_key.publickey()
        self._signer = PKCS1_v1_5.new(self._private_key)

    @property
    def identity(self):
        return binascii.hexlify(self._public_key.exportKey(format='DER')).decode('ascii')

class Transaction:
    def __init__(self, sender, recipient, value):
        self.sender = sender
        self.recipient = recipient
        self.value = value
        self.time = datetime.datetime.now()
```

Waiting for colab.research.google.com...

 BLOCK CHAIN pract 1 (A).ipynb ☆

File Edit View Insert Runtime Tools Help All changes saved

RAM

Disk

Editing

+

Code

+

Text

```
self.value = value
self.time = datetime.datetime.now()
def to_dict(self):
    if self.sender == "Genesis":
        identity = "Genesis"
    else:
        identity = self.sender.identity
    return collections.OrderedDict({
        'sender': identity,
        'recipient': self.recipient,
        'value': self.value,
        'time': self.time})
def sign_transaction(self):
    private_key = self.sender._private_key
    signer = PKCS1_v1_5.new(private_key)
    h = SHA.new(str(self.to_dict()).encode('utf8'))
    return binascii.hexlify(signer.sign(h)).decode('ascii')

Dinesh = Client()
Ramesh = Client()

t = Transaction(Dinesh,Ramesh.identity,5.0)

signature = t.sign_transaction()
print (signature)
```

656f0e37653d1bc3c538de01595a9c7e4a603e03740d1ed450f1ea32f568663cebff7ec9757cdd35d4f8b1a3524341528a48674981265b68a477c06d57271d6250f45d6e3946a4f8166fff50251956fa...

Waiting for colab.research.google.com...

Practical 2

Write the following programs for Blockchain in Python

- a) Create multiple transactions and display them.
- b) Create a blockchain, a genesis block and execute it.

→

```
def display_transaction(transaction):
    #for transaction in transactions:
    dict = transaction.to_dict()  print
    ("sender: " + dict['sender'])  print
    ('-----')
    print ("recipient: " + dict['recipient'])
    print ('-----')
    print ("value: " + str(dict['value']))
    print ('-----')
    print ("time: " + str(dict['time']))
    print ('-----') transactions
= []

Dinesh = Client()
Ramesh = Client()
Seema = Client()
Vijay = Client()

t1 = Transaction( Dinesh, Ramesh.identity, 15.0)
t1.sign_transaction() transactions.append(t1)

t2 = Transaction( Dinesh, Seema.identity, 6.0)
t2.sign_transaction()
transactions.append(t2)

t3 = Transaction( Ramesh, Vijay.identity, 2.0)
t3.sign_transaction() transactions.append(t3)
t4 = Transaction( Seema, Ramesh.identity, 4.0)
t4.sign_transaction()
transactions.append(t4)

t5 = Transaction( Vijay, Seema.identity, 7.0)
t5.sign_transaction()
transactions.append(t5)
```

```
t6 = Transaction( Ramesh, Seema.identity, 3.0)
t6.sign_transaction()
transactions.append(t6)
```

```
t7 = Transaction( Seema, Dinesh.identity, 8.0)
t7.sign_transaction()
transactions.append(t7)
```

```
t8 = Transaction( Seema, Ramesh.identity, 1.0)
t8.sign_transaction()
transactions.append(t8)
```

```
t9 = Transaction( Vijay, Dinesh.identity, 5.0)
t9.sign_transaction()
transactions.append(t9)
```

```
t10 = Transaction( Vijay, Ramesh.identity, 3.0)
t10.sign_transaction()
transactions.append(t10)
```

```
for transaction in transactions:
    display_transaction (transaction)    print
    ('-----')
```

```
class Block:
    def __init__(self):
        self.verified_transactions = []
    self.previous_block_hash = ""    self.Nonce
    = ""
```

```
last_block_hash = "" Dinesh
= Client()
t0 = Transaction ( "Genesis", Dinesh.identity, 500.0)
block0 = Block()
block0.previous_block_hash = None
Nonce = None
```



```

block0.verified_transactions.append (t0)
digest = hash (block0) last_block_hash
= digest

TPCoins = []

def dump_blockchain (self):
    print ("Number of blocks in the chain: " + str(len (self)))
    for x in range (len(TPCoins)):    block_temp =
TPCoins[x]    print ("block # " + str(x))    for
transaction in block_temp.verified_transactions:
        display_transaction (transaction)
        print ('-----')
    print ('=====')
TPCoins.append (block0) dump_blockchain(TPCoins)

```

Output:

```
[10] def display_transaction(transaction):  
    #for transaction in transactions:  
    dict = transaction.to_dict()  
    print ("sender: " + dict['sender'])  
    print ('-----')  
    print ("recipient: " + dict['recipient'])  
    print ('-----')  
    print ("value: " + str(dict['value']))  
    print ('-----')  
    print ("time: " + str(dict['time']))  
    print ('-----')
```

```
[11] transactions = []
```

```
[12] Dinesh = Client()  
    Ramesh = Client()  
    Seema = Client()  
    Vijay = Client()
```

```
[13] t1 = Transaction(  
    Dinesh,  
    Ramesh.identity,  
    15.0  
)
```

```
[15] t1.sign_transaction()  
    transactions.append(t1)
```

```
[16] t2 = Transaction(  
    Dinesh,  
    Seema.identity,  
    6.0  
)  
    t2.sign_transaction()  
    transactions.append(t2)  
    t3 = Transaction(  
    Ramesh,  
    Vijay.identity,  
    2.0  
)  
    t3.sign_transaction()  
    transactions.append(t3)  
    t4 = Transaction(  
    Seema,  
    Ramesh.identity,  
    4.0  
)  
    t4.sign_transaction()  
    transactions.append(t4)  
    t5 = Transaction(  
    Vijay,  
    Seema.identity,  
    7.0  
)  
    t5.sign_transaction()  
    transactions.append(t5)  
    t6 = Transaction(  
    Ramesh,  
    Seema.identity,  
    3.0  
)  
    t6.sign_transaction()  
    transactions.append(t6)  
    t7 = Transaction(  
    Seema,  
    Dinesh.identity,  
    8.0  
)  
    t7.sign_transaction()  
    transactions.append(t7)  
    t8 = Transaction(  
    Seema,  
    Ramesh.identity,  
    1.0  
)  
    t8.sign_transaction()  
    transactions.append(t8)  
    t9 = Transaction(  
    Vijay,  
    Dinesh.identity,  
    5.0  
)  
    t9.sign_transaction()  
    transactions.append(t9)
```

```

[16] t8.sign_transaction()
      transactions.append(t8)
      t9 = Transaction(
            Vijay,
            Dinesh.identity,
            5.0
        )
      t9.sign_transaction()
      transactions.append(t9)
      t10 = Transaction(
            Vijay,
            Ramesh.identity,
            3.0
        )
      t10.sign_transaction()
      transactions.append(t10)

[17] for transaction in transactions:
      display_transaction(transaction)
      print('-----')

sender: 30819f300d06092a864886f70d010101050003818d0030818d0030818902818100b5c36acef717c85079d5
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100b5c36acef717c85079d5
-----
value: 15.0
-----
time: 2021-05-05 07:37:46.751762
-----
sender: 30819f300d06092a864886f70d010101050003818d003081890281810096e3e436b160bf2feecba6d
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100b5c36acef717c85079d5
-----
value: 15.0
-----
time: 2021-05-05 07:37:46.751762
-----
sender: 30819f300d06092a864886f70d010101050003818d003081890281810096e3e436b160bf2feecba6d
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100a3b073014e3d9a1e3353
-----
value: 6.0
-----
time: 2021-05-05 07:38:17.042489
-----
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100b5c36acef717c85079d5eb8
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100967bf6965c7e25f7c711
-----
value: 2.0
-----
time: 2021-05-05 07:38:17.044049
-----
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100a3b073014e3d9a1e335365f
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100b5c36acef717c85079d5
-----
value: 4.0
-----
time: 2021-05-05 07:38:17.045503
-----
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100967bf6965c7e25f7c711a30
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100a3b073014e3d9a1e3353
-----
value: 7.0
-----
time: 2021-05-05 07:38:17.046078
-----

[18] class Block:
      def __init__(self):
          self.verified_transactions = []
          self.previous_block_hash = ""
          self.Nonce = ""

[19] last_block_hash = ""

```



```

[20] Dinesh = Client()

[21] t0 = Transaction (
    "Genesis",
    Dinesh.identity,
    500.0
)

[22] block0 = Block()

[23] block0.previous_block_hash = None
    Nonce = None

[24] block0.verified_transactions.append (t0)

[25] digest = hash (block0)
    last_block_hash = digest

[26] def dump_blockchain (self):
    print ("Number of blocks in the chain: " + str(len (self)))
    for x in range (len(TPCoins)):
        block_temp = TPCoins[x]
        print ("block # " + str(x))
        for transaction in block_temp.verified_transactions:
            display_transaction (transaction)
            print ('-----')
        print ('=====')

[27] TPCoins = []

[30] def dump_blockchain (self):
    print ("Number of blocks in the chain: " + str(len (self)))
    for x in range (len(TPCoins)):
        block_temp = TPCoins[x]
        print ("block # " + str(x))
        for transaction in block_temp.verified_transactions:
            display_transaction (transaction)
            print ('-----')
        print ('=====')

[31] TPCoins.append (block0)

[32] dump_blockchain(TPCoins)

Number of blocks in the chain: 1
block # 0
sender: Genesis
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100df7d100622cc76eab161
-----
value: 500.0
-----
time: 2021-05-05 07:41:01.767902
-----
=====

```

Practical 3

Write the following programs for Blockchain in Python:

- a) Create a mining function and test it.
- b) Add blocks to the miner and dump the blockchain.

→

```
def sha256(message):
    return hashlib.sha256(message.encode('ascii')).hexdigest()

def mine(message, difficulty=1):
    assert difficulty >= 1
    prefix = '1' * difficulty
    for i in range(1000):
        digest = sha256(str(hash(message)) + str(i))
        if digest.startswith(prefix):
            print("after " + str(i) + " iterations found nonce: " + digest)
            return digest
    last_transaction_index = 0
    block = Block()
    for i in range(3):
        temp_transaction = transactions[last_transaction_index]
        # validate transaction
        # if valid
        block.verified_transactions.append(temp_transaction)
        last_transaction_index += 1
    mine("test message", 2)

block.previous_block_hash = last_block_hash
block.Nonce = mine(block, 2)
digest = hash(block)
TPCoins.append(block)
last_block_hash = digest

# Miner 2 adds a block
block = Block()

for i in range(3):
    temp_transaction = transactions[last_transaction_index]

    # validate transaction
    # if valid
    block.verified_transactions.append(temp_transaction)
    last_transaction_index += 1
    block.previous_block_hash = last_block_hash
    block.Nonce = mine(block, 2)
    digest = hash(block)
    TPCoins.append(block)
    last_block_hash = digest
# Miner 3 adds a block
block = Block()
```

```
for i in range(3): temp_transaction =
transactions[last_transaction_index]
    #display_transaction (temp_transaction)
    # validate transaction
# if valid
    block.verified_transactions.append (temp_transaction)
last_transaction_index += 1

block.previous_block_hash = last_block_hash block.Nonce
= mine (block, 2)
digest = hash (block)

TPCoins.append (block) last_block_hash
= digest
dump_blockchain(TPCoins)
```

Full Output:

```

▶ # import libraries
import hashlib
import random
import string
import json
import binascii
import numpy as np
import pandas as pd
import pylab as pl
import logging
import datetime
import collections

[ ] pip install pycryptodome

Collecting pycryptodome
  Downloading https://files.pythonhosted.org/packages/ad/16/9627ab0493894a11c68e46000dbcc
    | 1.9MB 5.6MB/s
Installing collected packages: pycryptodome
Successfully installed pycryptodome-3.10.1

[ ] # following imports are required by PKI
import Crypto
import Crypto.Random
from Crypto.Hash import SHA
from Crypto.PublicKey import RSA
from Crypto.Signature import PKCS1_v1_5

[ ] import binascii
class Client:
    def __init__(self):
        random = Crypto.Random.new().read
        self._private_key = RSA.generate(1024, random)
        self._public_key = self._private_key.publickey()
        self._signer = PKCS1_v1_5.new(self._private_key)

    @property
    def identity(self):
        return binascii.hexlify(self._public_key.exportKey(format='DER')).decode('ascii')

class Transaction:
    def __init__(self, sender, recipient, value):
        self.sender = sender
        self.recipient = recipient
        self.value = value
        self.time = datetime.datetime.now()
    def to_dict(self):
        if self.sender == "Genesis":
            identity = "Genesis"
        else:
            identity = self.sender.identity
        return collections.OrderedDict({
            'sender': identity,
            'recipient': self.recipient,
            'value': self.value,
            'time': self.time})
    def sign_transaction(self):
        private_key = self.sender._private_key
        signer = PKCS1_v1_5.new(private_key)
        h = SHA.new(str(self.to_dict()).encode('utf8'))
        return binascii.hexlify(signer.sign(h)).decode('ascii')

[ ]

[ ] Dinesh = Client()
Ramesh = Client()

[ ] t = Transaction(Dinesh, Ramesh.identity, 5.0)

[ ] signature = t.sign_transaction()
print (signature)

251f28f6f523733739979611b404c755210b5b6a70f28d5eb3e3049f7dceea873e472f0df41a55152c71bb6f5

[ ] def display_transaction(transaction):
    #for transaction in transactions:
    dict = transaction.to_dict()
    print ("sender: " + dict['sender'])
    print ('-----')

```

```
[ ] def display_transaction(transaction):
    #for transaction in transactions:
    dict = transaction.to_dict()
    print ("sender: " + dict['sender'])
    print ('-----')
    print ('recipient: " + dict['recipient'])
    print ('-----')
    print ("value: " + str(dict['value']))
    print ('-----')
    print ("time: " + str(dict['time']))
    print ('-----')
```

```
[ ] transactions = []
```

```
[ ] Dinesh = Client()
    Ramesh = Client()
    Seema = Client()
    Vijay = Client()
```

```
[ ] t1 = Transaction(
    Dinesh,
    Ramesh.identity,
    15.0
)
```

```
[ ] t1.sign_transaction()
    transactions.append(t1)
```

```
[ ] t2 = Transaction(
    Dinesh,
    Seema.identity,
    6.0
)
t2.sign_transaction()
transactions.append(t2)
t3 = Transaction(
    Ramesh,
    Vijay.identity,
    2.0
)
t3.sign_transaction()
transactions.append(t3)
t4 = Transaction(
    Seema,
    Ramesh.identity,
    4.0
)
t4.sign_transaction()
transactions.append(t4)
t5 = Transaction(
    Vijay,
    Seema.identity,
    7.0
)
t5.sign_transaction()
transactions.append(t5)
t6 = Transaction(
    Ramesh,
    Seema.identity,
    3.0
)
t6.sign_transaction()
transactions.append(t6)
t7 = Transaction(
    Seema,
    Dinesh.identity,
    8.0
)
t7.sign_transaction()
transactions.append(t7)
t8 = Transaction(
    Seema,
    Ramesh.identity,
    1.0
)
t8.sign_transaction()
transactions.append(t8)
t9 = Transaction(
    Vijay,
    Dinesh.identity,
    5.0
)
t9.sign_transaction()
transactions.append(t9)
t10 = Transaction(
    Vijay,
    Ramesh.identity,
    3.0
)
t10.sign_transaction()
transactions.append(t10)
```



```
[ ] for transaction in transactions:
    display_transaction(transaction)
    print('-----')

sender: 30819f300d06092a864886f70d010101050003818d00308189028181008f5e89df98216eb2c1e4713
-----
recipient: 30819f300d06092a864886f70d010101050003818d003081890281810082e40f44f9cef42cac49
-----
value: 15.0
-----
time: 2021-04-08 06:10:52.172517
-----
sender: 30819f300d06092a864886f70d010101050003818d00308189028181008f5e89df98216eb2c1e4713
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100a0b9b51bb0c81cbbad86
-----
value: 6.0
-----
time: 2021-04-08 06:11:40.567785
-----
sender: 30819f300d06092a864886f70d010101050003818d003081890281810082e40f44f9cef42cac492d1
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100d776af73071682cb494e
-----
value: 2.0
-----
time: 2021-04-08 06:11:40.569278
-----
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100a0b9b51bb0c81cbbad86384
-----
recipient: 30819f300d06092a864886f70d010101050003818d003081890281810082e40f44f9cef42cac49
-----
value: 4.0
-----
time: 2021-04-08 06:11:40.570658
-----
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100d776af73071682cb494e752
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100a0b9b51bb0c81cbbad86
-----
value: 7.0
-----
time: 2021-04-08 06:11:40.572173
-----
sender: 30819f300d06092a864886f70d010101050003818d003081890281810082e40f44f9cef42cac492d1
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100a0b9b51bb0c81cbbad86
-----
value: 3.0
time: 2021-04-08 06:11:40.574000
```

```
[ ] class Block:
    def __init__(self):
        self.verified_transactions = []
        self.previous_block_hash = ""
        self.Nonce = ""

[ ] last_block_hash = ""

[ ] Dinesh = Client()

[ ] t0 = Transaction (
    "Genesis",
    Dinesh.identity,
    500.0
)

[ ] block0 = Block()

[ ] block0.previous_block_hash = None
    Nonce = None

[ ] block0.verified_transactions.append (t0)
```

```
[ ] digest = hash (block0)
    last_block_hash = digest

[ ] def dump_blockchain (self):
    print ("Number of blocks in the chain: " + str(len (self)))
    for x in range (len(TPCoins)):
        block_temp = TPCoins[x]
        print ("block # " + str(x))
        for transaction in block_temp.verified_transactions:
            display_transaction (transaction)
            print ('-----')
        print ('=====')

[ ] TPCoins = []

[ ] def dump_blockchain (self):
    print ("Number of blocks in the chain: " + str(len (self)))
    for x in range (len(TPCoins)):
        block_temp = TPCoins[x]
        print ("block # " + str(x))
        for transaction in block_temp.verified_transactions:
            display_transaction (transaction)
            print ('-----')
        print ('=====')

[ ] TPCoins.append (block0)

[ ] dump_blockchain(TPCoins)

Number of blocks in the chain: 1
block # 0
sender: Genesis
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100bae5c5a46a1591af94c0
-----
value: 500.0
-----
time: 2021-04-08 06:16:28.464142
-----
=====

[ ] def sha256(message):
    return hashlib.sha256(message.encode('ascii')).hexdigest()

[ ] def mine(message, difficulty=1):
    assert difficulty >= 1
    prefix = '1' * difficulty
    for i in range(1000):
        digest = sha256(str(hash(message)) + str(i))
        if digest.startswith(prefix):
            print ("after " + str(i) + " iterations found nonce: " + digest)
            return digest

[ ] mine ("test message", 2)

'938c72d2197fa6c2294df7bc8cea6be98769aad888e3cfa15558c78469394ebd'

[ ] last_transaction_index = 0

[ ] block = Block()
    for i in range(3):
        temp_transaction = transactions[last_transaction_index]
        # validate transaction
        # if valid
        block.verified_transactions.append (temp_transaction)
        last_transaction_index += 1

    block.previous_block_hash = last_block_hash
    block.Nonce = mine (block, 2)
    digest = hash (block)
    TPCoins.append (block)
    last_block_hash = digest
```

```
+ Code + Text Copy to Drive Connect Editing ^

[ ] # Miner 2 adds a block
block = Block()

for i in range(3):
    temp_transaction = transactions[last_transaction_index]
    # validate transaction
    # if valid
    block.verified_transactions.append (temp_transaction)
    last_transaction_index += 1
block.previous_block_hash = last_block_hash
block.Nonce = mine(block, 2)
digest = hash (block)
TPCoins.append (block)
last_block_hash = digest
# Miner 3 adds a block
block = Block()

for i in range(3):
    temp_transaction = transactions[last_transaction_index]
    #display_transaction (temp_transaction)
    # validate transaction
    # if valid
    block.verified_transactions.append (temp_transaction)
    last_transaction_index += 1

block.previous_block_hash = last_block_hash
block.Nonce = mine (block, 2)
digest = hash (block)

TPCoins.append (block)
last_block_hash = digest

[ ] dump_blockchain(TPCoins)

time: 2021-04-08 06:11:40.570658
-----
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100d776af73071682cb494e752
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100a0b9b51bb0c81cbbad86
-----
value: 7.0
-----
time: 2021-04-08 06:11:40.572173
-----
sender: 30819f300d06092a864886f70d010101050003818d003081890281810082e40f44f9cef42cac492d1
-----
recipient: 30819f300d06092a864886f70d010101050003818d0030818902818100a0b9b51bb0c81cbbad86
-----
value: 3.0
-----
time: 2021-04-08 06:11:40.574036
-----
=====
block # 3
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100a0b9b51bb0c81cbbad86384
-----
recipient: 30819f300d06092a864886f70d010101050003818d00308189028181008f5e89df98216eb2c1e4
-----
value: 8.0
-----
time: 2021-04-08 06:11:40.575310
-----
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100a0b9b51bb0c81cbbad86384
-----
recipient: 30819f300d06092a864886f70d010101050003818d003081890281810082e40f44f9cef42cac49
-----
value: 1.0
-----
time: 2021-04-08 06:11:40.576645
-----
sender: 30819f300d06092a864886f70d010101050003818d0030818902818100d776af73071682cb494e752
-----
recipient: 30819f300d06092a864886f70d010101050003818d00308189028181008f5e89df98216eb2c1e4
-----
value: 5.0
-----
time: 2021-04-08 06:11:40.578007
-----
=====
```

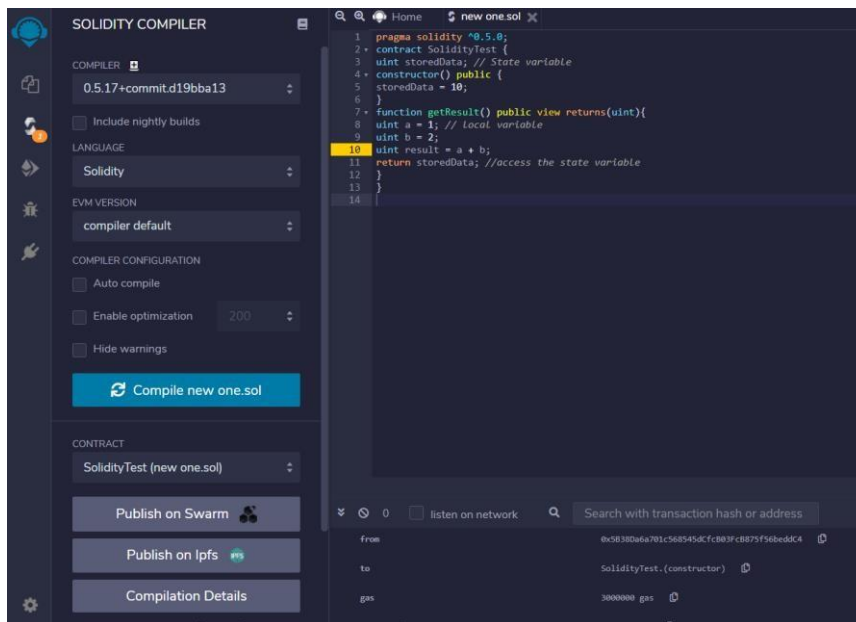
Practical 4

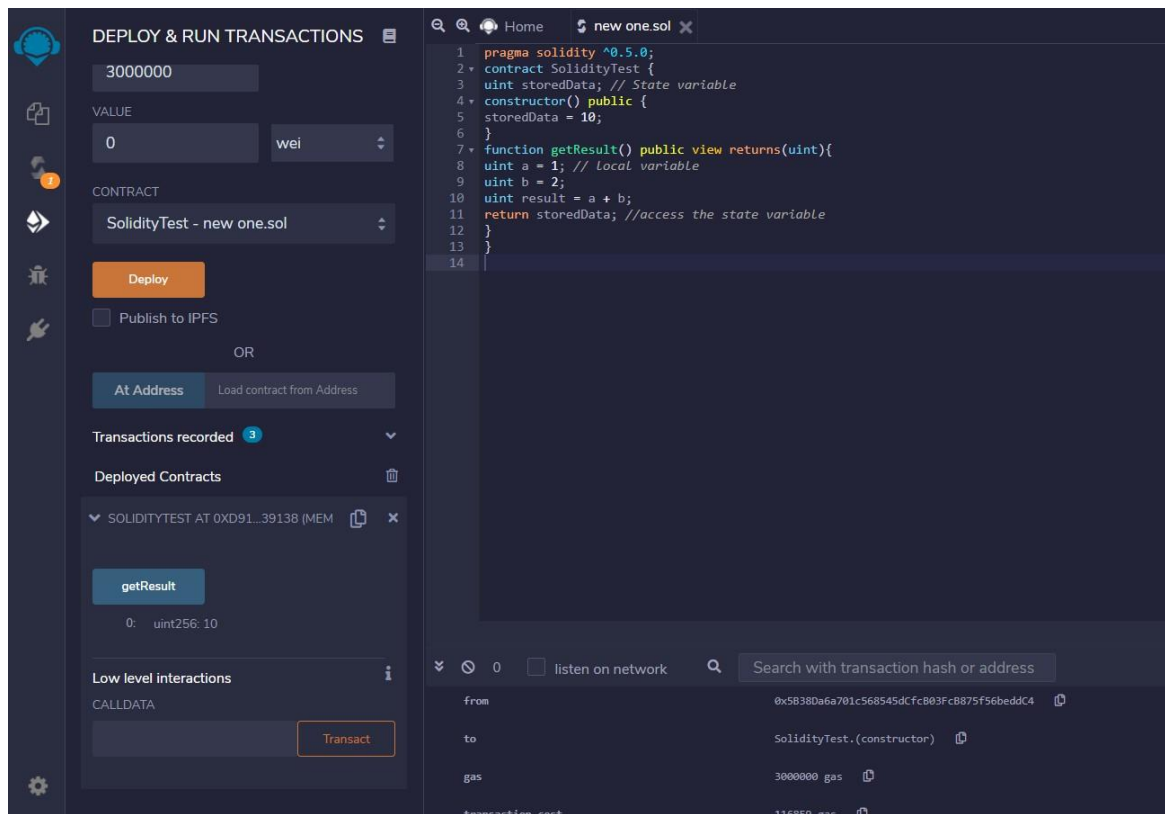
Implement and demonstrate the use of the following in Solidity:

- a) Variable
- b) Operators
- c) Loops
- d) Decision Making
- e) Strings

a.Variable pragma solidity ^0.5.0;
contract SolidityTest { uint
storedData; // State variable
constructor() public {
storedData = 10;
}
function getResult() public view returns(uint){
uint a = 1; // local variable
uint b = 2; uint result = a
+ b;
return storedData; //access the state variable
}
}

Output:





// Solidity program to demonstrate state variables

```
pragma solidity ^0.5.0; // Creating a contract
contract Solidity_var_Test { // Declaring a state
variable uint8 public state_var; // Defining a
constructor constructor() public { state_var = 16;
}
}
```

Output:



COMPILER

0.5.17+commit.d19bba13

☐ Include nightly builds

LANGUAGE

Solidity

EVM VERSION

compiler default

COMPILER CONFIGURATION

☐ Auto compile

☐ Enable optimization 200

☐ Hide warnings

Compile new one.sol

CONTRACT

Solidity_var_Test (new one.sol)

Publish on Swarm

Publish on Ipfs

Compilation Details

Home new one.sol

```
1 // Solidity program to demonstrate state variables
2 pragma solidity ^0.5.0;
3
4 // Creating a contract
5 contract Solidity_var_Test {
6
7 // Declaring a state variable
8 uint8 public state_var;
9
10 // Defining a constructor
11 constructor() public {
12 state_var = 16;
13 }
14 }
15
```

0 listen on network

Search with transaction hash or

from	0x5838Da6a701c568545dCfcB03FcB875
to	SolidityTest.(constructor)
gas	3000000 gas

Page 22 | 68



SOLIDITY COMPILER

COMPILER 
0.5.17+commit.d19bba13 

☐ Include nightly builds

LANGUAGE
Solidity 

EVM VERSION
compiler default 

COMPILER CONFIGURATION

☐ Auto compile

☐ Enable optimization 200 

☐ Hide warnings

Compile Variable.sol 

CONTRACT
Solidity_var_Test (Variable.sol) 

Publish on Swarm 

Publish on Ipfs 

Compilation Details

ABI  Bytecode 

Home Variable.sol 

```
1 // Solidity program to
2 // demonstrate state
3 // variables
4 pragma solidity ^0.5.0;
5
6 // Creating a contract
7 contract Solidity_var_Test {
8
9 // Declaring a state variable
10 uint8 public state_var;
11
12 // Defining a constructor
13 constructor() public {
14 state_var = 16;
15 }
16 }
```



DEPLOY & RUN TRANSACTIONS

ENVIRONMENT
JavaScript VM 

ACCOUNT 
0x5B3...eddC4 (99.9999999€  

GAS LIMIT
3000000

VALUE
0 wei 

CONTRACT
Solidity_var_Test - Variable.sol 

Deploy

☐ Publish to IPFS

OR

At Address  Load contract from Address

Transactions recorded 1 

Deployed Contracts 

SOLIDITY_VAR_TEST AT 0XD91...39138  

state_var

0: uint8: 16

Low level interactions 

CALLDATA

Transact

Home Variable.sol 

```
1 // Solidity program to
2 // demonstrate state
3 // variables
4 pragma solidity ^0.5.0;
5
6 // Creating a contract
7 contract Solidity_var_Test {
8
9 // Declaring a state variable
10 uint8 public state_var;
11
12 // Defining a constructor
13 constructor() public {
14 state_var = 16;
15 }
16 }
```

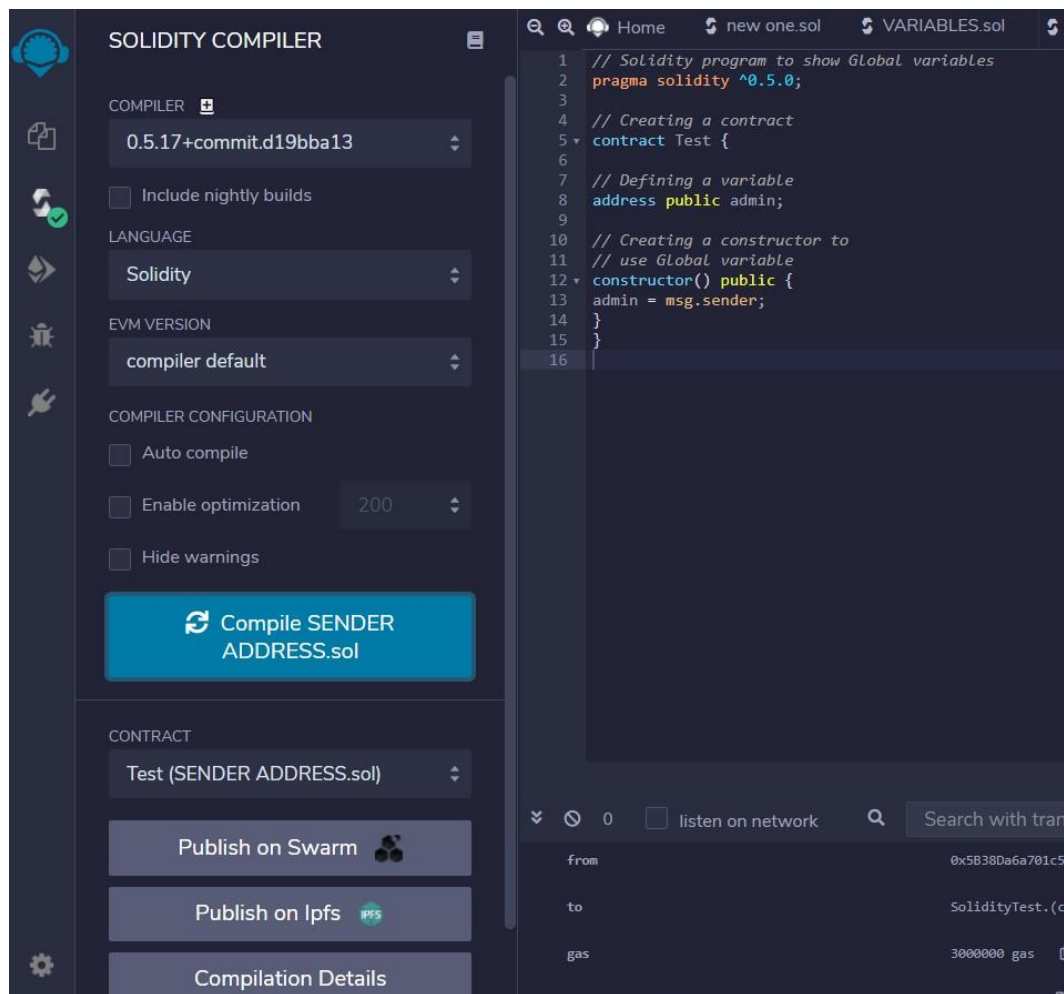

// Solidity program to show Global variables

```
pragma solidity ^0.5.0; //  
Creating a contract  
contract Test {
```

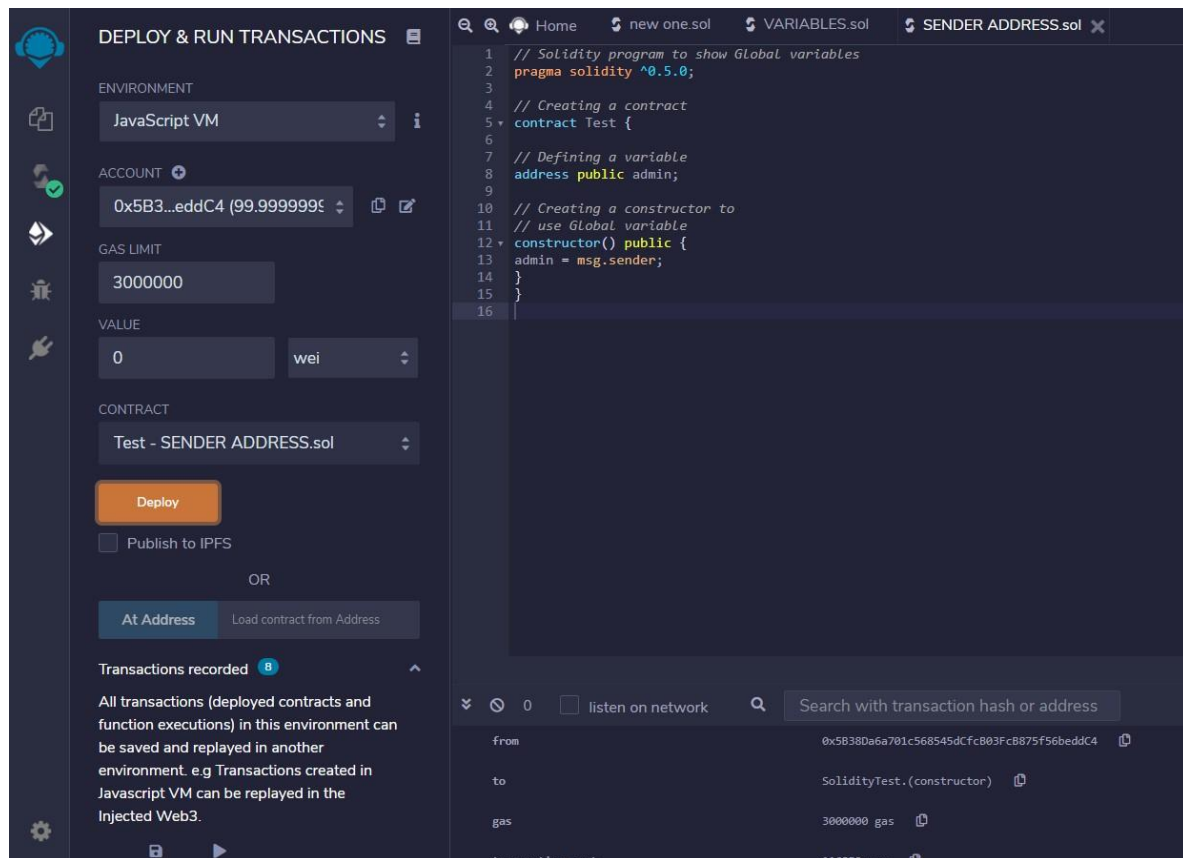
```
// Defining a variable  
address public admin;
```

```
// Creating a constructor to  
// use Global variable  
constructor() public { admin  
= msg.sender;  
}  
}
```

Output:



The screenshot displays the Solidity Compiler web interface. On the left, the 'COMPILER' section shows the version '0.5.17+commit.d19bba13', with options for 'Include nightly builds', 'LANGUAGE' set to 'Solidity', and 'EVM VERSION' set to 'compiler default'. Under 'COMPILER CONFIGURATION', there are checkboxes for 'Auto compile', 'Enable optimization' (set to 200), and 'Hide warnings'. A large blue button labeled 'Compile SENDER ADDRESS.sol' is present. Below this, the 'CONTRACT' section shows 'Test (SENDER ADDRESS.sol)' selected, with buttons for 'Publish on Swarm', 'Publish on Ipfs', and 'Compilation Details'. The main editor on the right shows the Solidity code from the previous blocks, with line numbers 1 through 16. The bottom of the interface shows a transaction summary with fields for 'from', 'to', and 'gas'.



b.Operators

// Solidity contract to demonstrate Arithmetic Operator
pragma solidity ^0.5.0;

// Creating a contract
contract SolidityTest {

// Initializing variables
uint16 public a = 20;
uint16 public b = 10;

// Initializing a variable with sum uint
public sum = a + b;

// Initializing a variable with the difference uint
public diff = a - b;

// Initializing a variable with product
uint public mul = a * b;

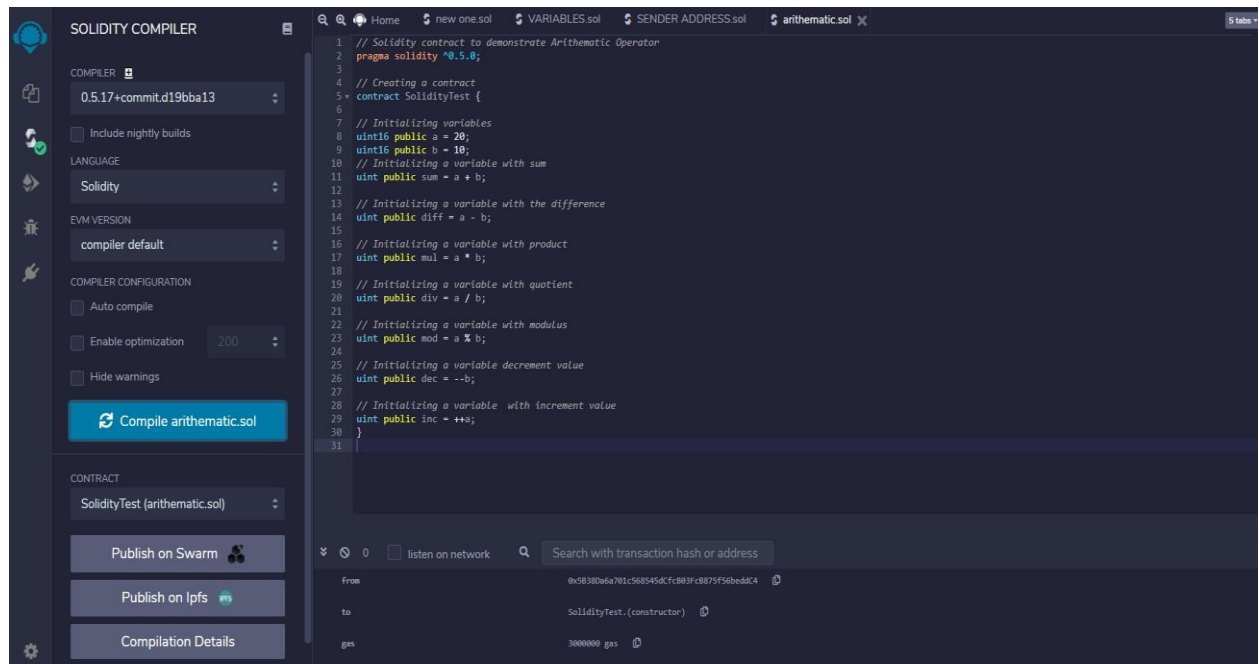
```
// Initializing a variable with quotient uint
public div = a / b;
```

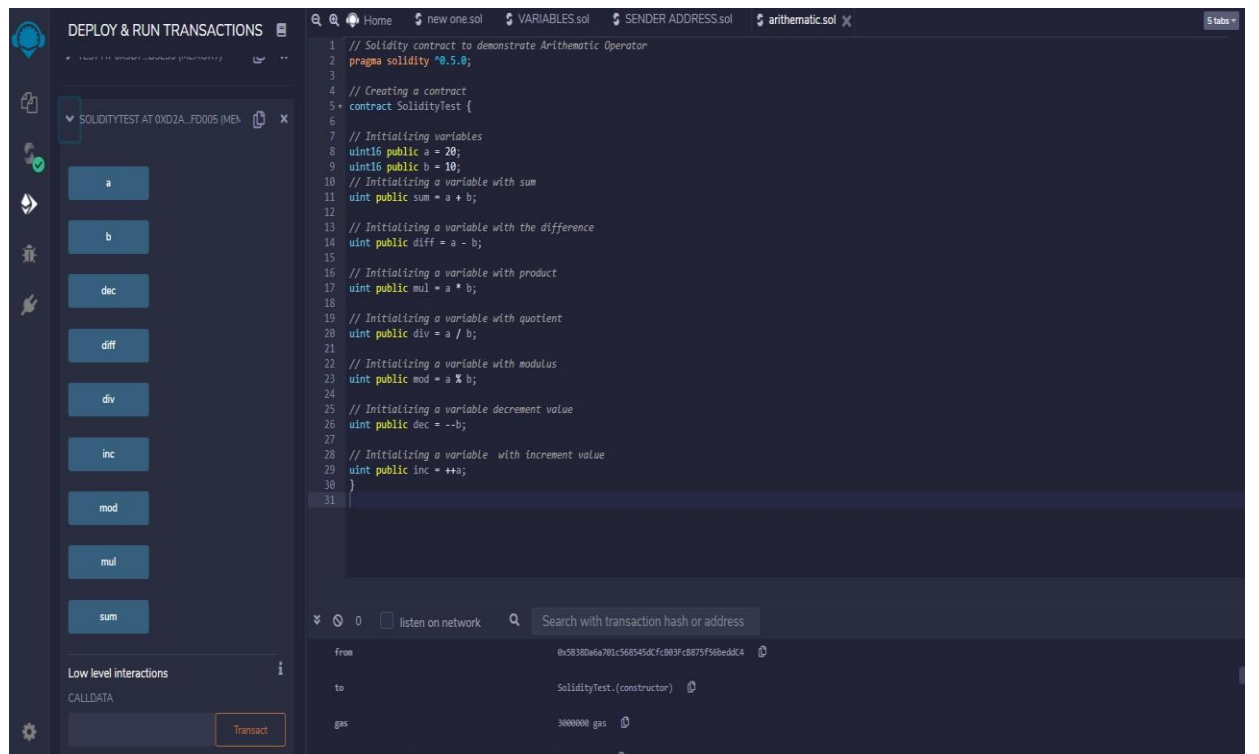
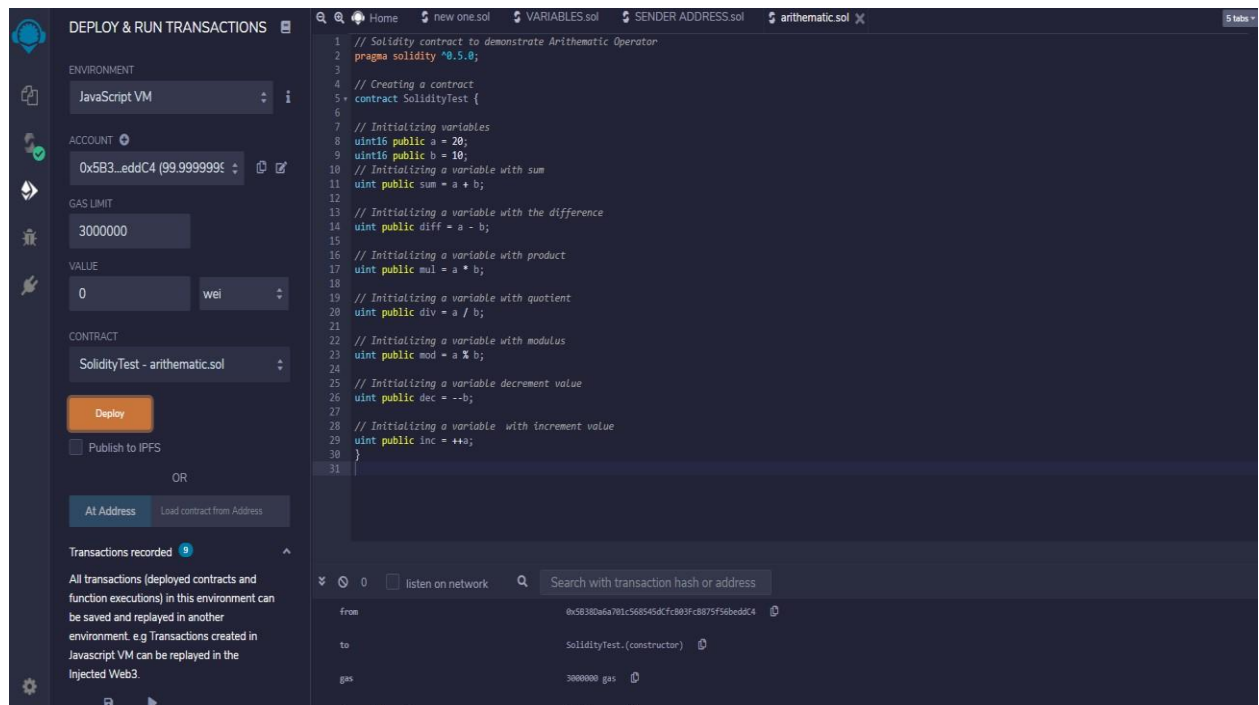
```
// Initializing a variable with modulus uint
public mod = a % b;
```

```
// Initializing a variable decrement value uint
public dec = --b;
```

```
// Initializing a variable with increment value uint
public inc = ++a;
}
```

Output:





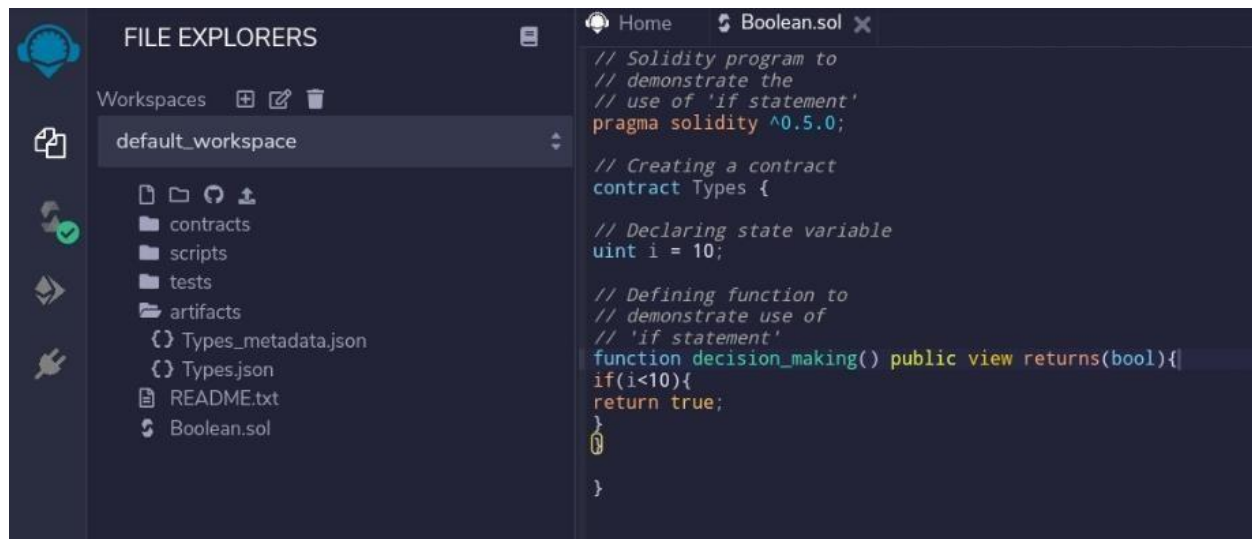
C.Decision Making

// Solidity program to demonstrate the use of 'if statement' pragma solidity ^0.5.0;

```
// Creating a contract contract
Types {
// Declaring state variable uint
i = 10;

// Defining function to demonstrate use of 'if statement'
function decision_making() public view returns(bool){
if(i<10){ return true;
}
}
}
```

Output:

A screenshot of a Solidity IDE interface. On the left, the 'FILE EXPLORERS' panel shows a workspace named 'default_workspace' containing folders for 'contracts', 'scripts', 'tests', and 'artifacts', along with files 'Types_metadata.json', 'Types.json', 'README.txt', and 'Boolean.sol'. The main editor area on the right displays the Solidity code from the previous block, with syntax highlighting. The code defines a contract named 'Types' with a state variable 'i' of type 'uint' set to 10, and a public view function 'decision_making()' that returns a boolean value based on whether 'i' is less than 10. The code is as follows:

```
// Solidity program to
// demonstrate the
// use of 'if statement'
pragma solidity ^0.5.0;

// Creating a contract
contract Types {

// Declaring state variable
uint i = 10;

// Defining function to
// demonstrate use of
// 'if statement'
function decision_making() public view returns(bool){
if(i<10){
return true;
}
}
}
```



SOLIDITY COMPILER

COMPILER 

0.5.17+commit.d19bba13 

☐ Include nightly builds

LANGUAGE

Solidity 



EVM VERSION

compiler default 

COMPILER CONFIGURATION

☐ Auto compile

☐ Enable optimization 200 

☐ Hide warnings

 Compile Boolean.sol

CONTRACT

Types (Boolean.sol) 

Publish on Swarm 

Publish on Ipfs 

Compilation Details

 ABI  Bytecode

Home Boolean.sol 

```
// Solidity program to
// demonstrate the
// use of 'if statement'
pragma solidity ^0.5.0;

// Creating a contract
contract Types {

// Declaring state variable
uint i = 10;

// Defining function to
// demonstrate use of
// 'if statement'
function decision_making() public view returns(bool){
    if(i<10){
        return true;
    }
}
```

The screenshot displays the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar is active, showing the deployment configuration for the 'Boolean.sol' contract. The environment is set to 'JavaScript VM', the account is '0x5B3...eddC4 (99.9999999%)', the gas limit is '3000000', and the value is '0 wei'. The contract is 'Types - Boolean.sol'. Below the deployment options, it shows 'Transactions recorded' (1) and 'Deployed Contracts' (Types AT 0xD91...39138 (MEMORY)). The 'Low level interactions' section shows the 'CALLDATA' field and a 'Transact' button.

On the right, the 'Boolean.sol' contract code is displayed in a dark-themed editor. The code is a Solidity program demonstrating the use of an 'if' statement. It includes a pragma statement for Solidity version 0.5.0, a contract definition for 'Types', a state variable 'i' of type 'uint' initialized to 10, and a public view function 'decision_making()' that returns a boolean value based on the value of 'i'.

```
// Solidity program to
// demonstrate the
// use of 'if statement'
pragma solidity ^0.5.0;

// Creating a contract
contract Types {

// Declaring state variable
uint i = 10;

// Defining function to
// demonstrate use of
// 'if statement'
function decision_making() public view returns(bool){
    if(i<10){
        return true;
    }
}
```

// Solidity program to demonstrate the use of 'if...else' statement pragma
solidity ^0.5.0;

```

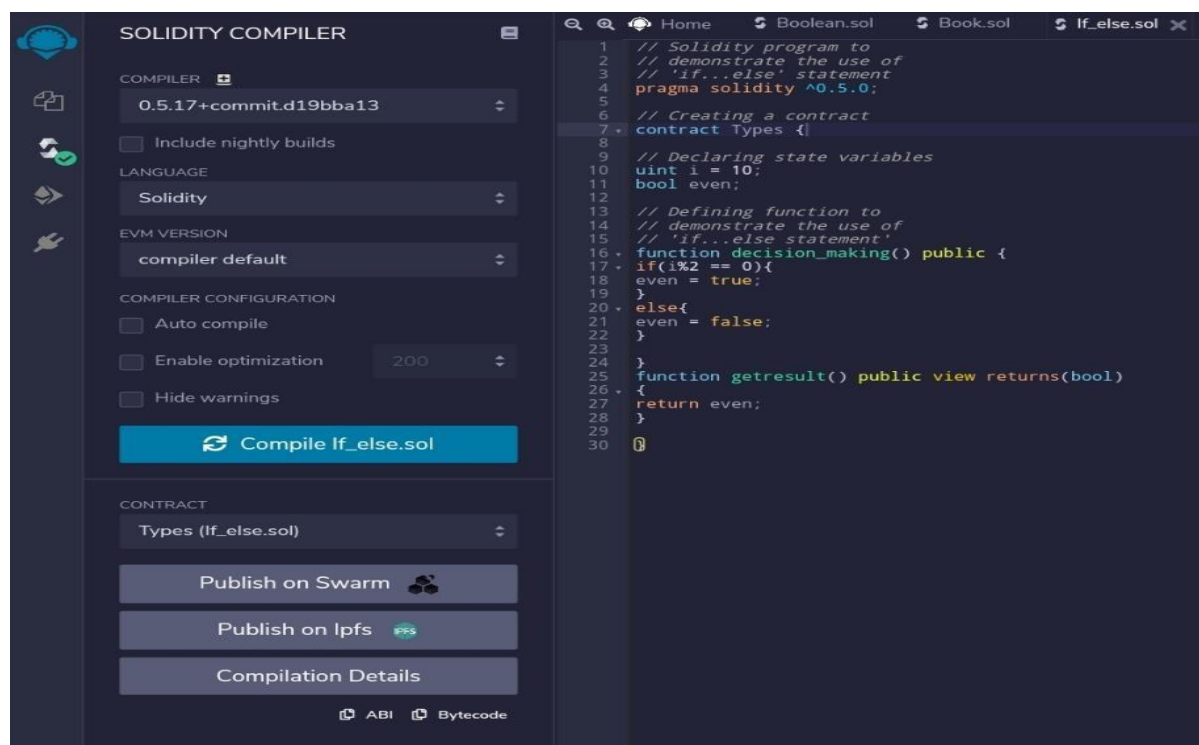
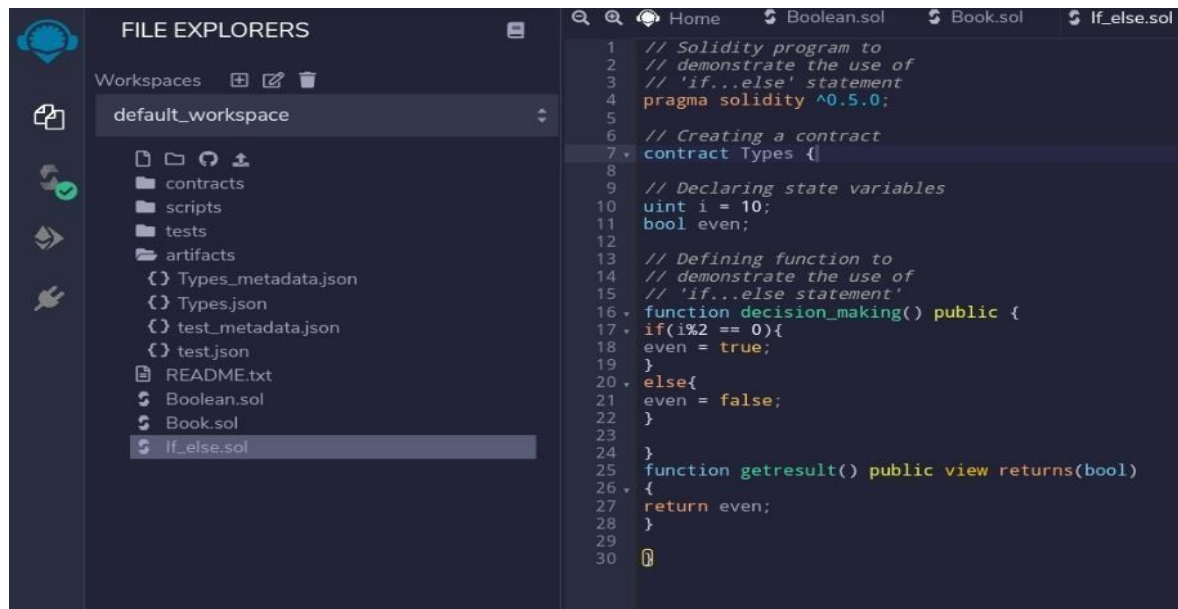
// Creating a contract contract
Types {

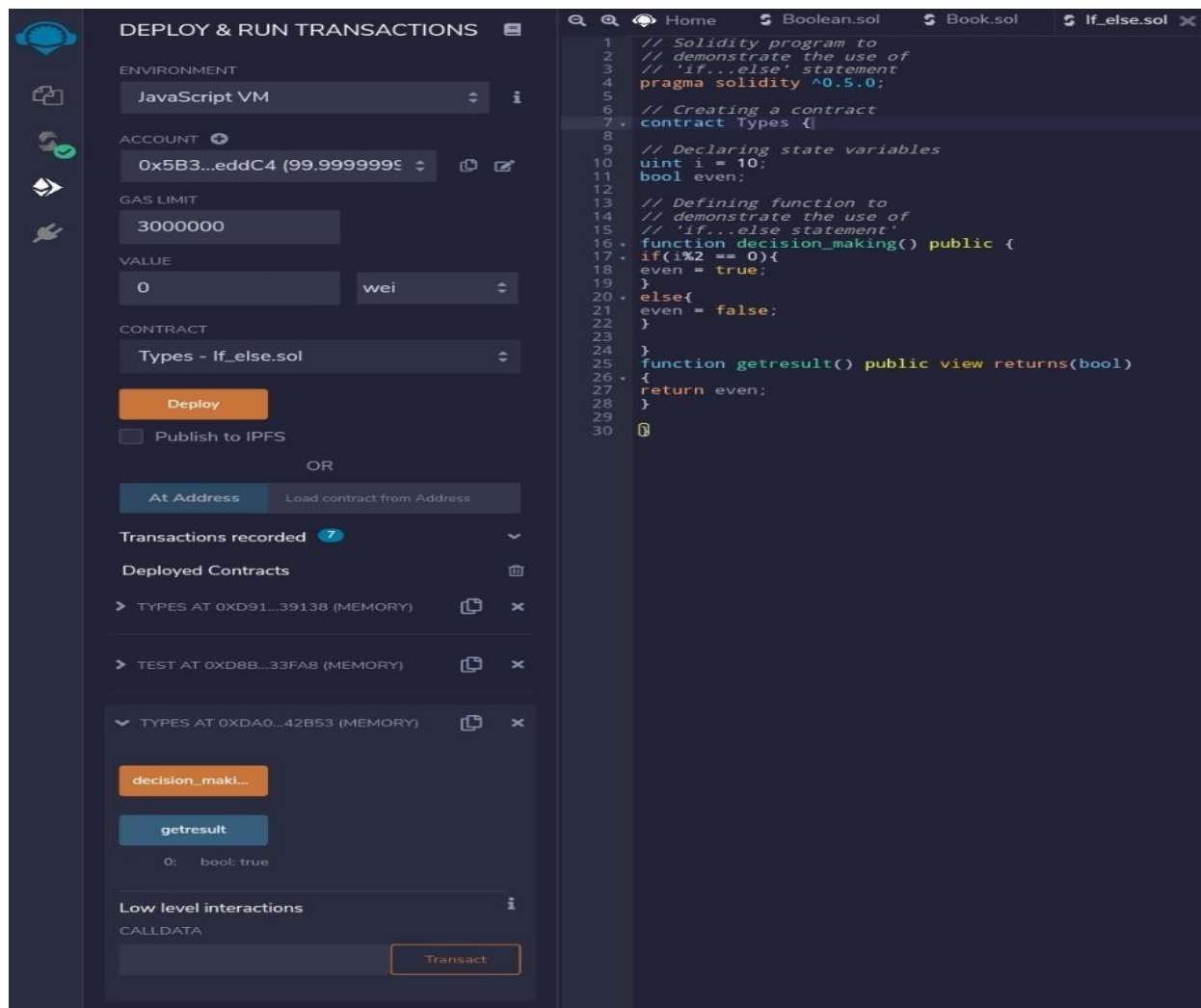
// Declaring state variables
uint i = 10; bool even;

// Defining function to
// demonstrate the use of
// 'if...else statement' function
decision_making() public { if(i%2
== 0){ even = true; } else{ even =
false;
} }
function getResult() public view returns(bool)
{ return
even;
}
}

```

Output:





(III) Strings

// Solidity program to demonstrate

// how to create a contract pragma

solidity ^0.4.23;

// Creating a contract contract

Test {

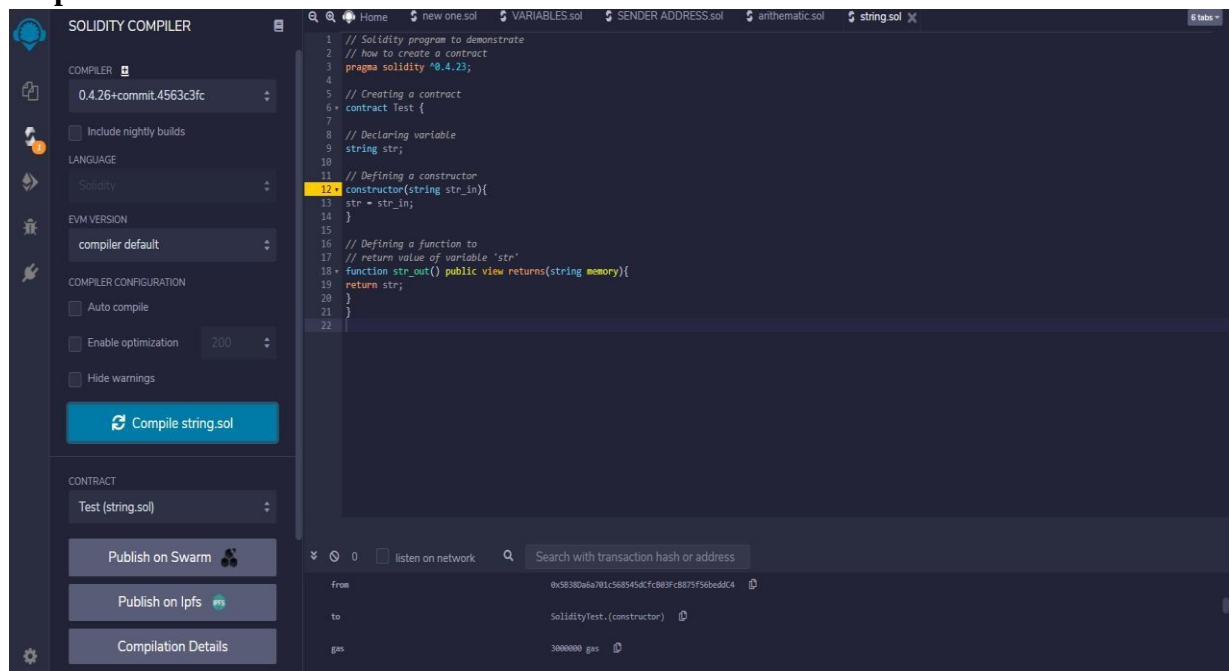
// Declaring variable string

str;

```
// Defining a constructor
constructor(string str_in){
    str = str_in;
}
```

```
// Defining a function to
// return value of variable 'str'
function str_out() public view returns(string memory){
    return str;
}
}
```

Output



DEPLOY & RUN TRANSACTIONS

SOLIDITY_VAR_TEST AT 0X358...D5EE3

state_var

Low level interactions

CALLDATA

0

Transact

The calldata should be a valid hexadecimal value.

TEST AT 0X307...B5E39 (MEMORY)

TEST AT 0XD2A...FD005 (MEM)

TEST AT 0XDDA...5482D (MEMORY)

str_out

Low level interactions

CALLDATA

Transact

1 // Solidity program to demonstrate

2 // how to create a contract

3 pragma solidity ^0.4.23;

4

5 // Creating a contract

6 contract Test {

7

8 // Declaring variable

9 string str;

10

11 // Defining a constructor

12 constructor(string str_in){

13 str = str_in;

14 }

15

16 // Defining a function to

17 // return value of variable 'str'

18 function str_out() public view returns(string memory){

19 return str;

20 }

21 }

22 }

listen on network

Search with transaction hash or address

from 0x5B38D6a781c568545cF803F0887F56B6dC4

to SolidityTest.constructor

gas 3000000 gas

Practical 5

Implement and demonstrate the use of the following in Solidity:

- a)** Arrays
- b)** Enums
- c)** Structs
- d)** Mappings
- e)** Conversations
- f)** Ether Units
- g)** Special Variables

(I).Arrays

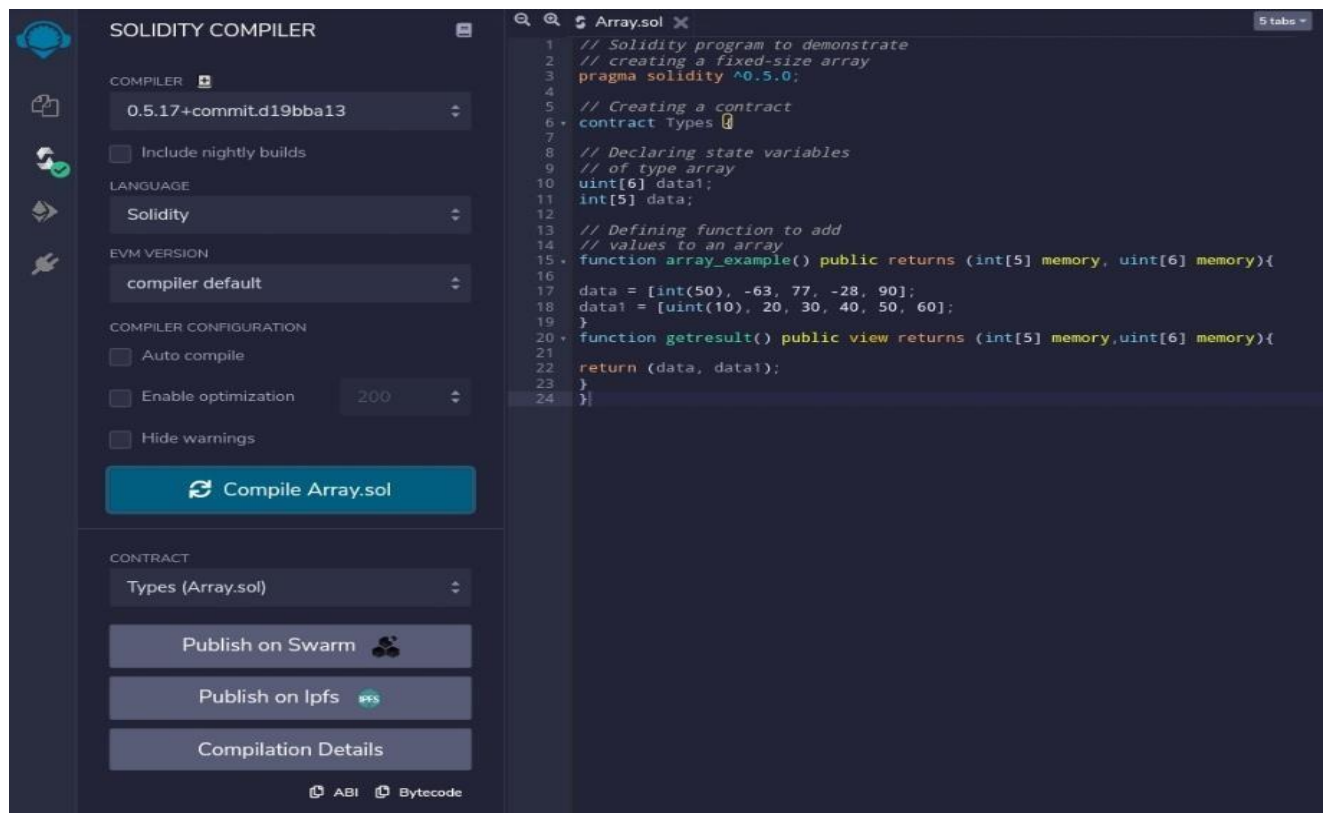
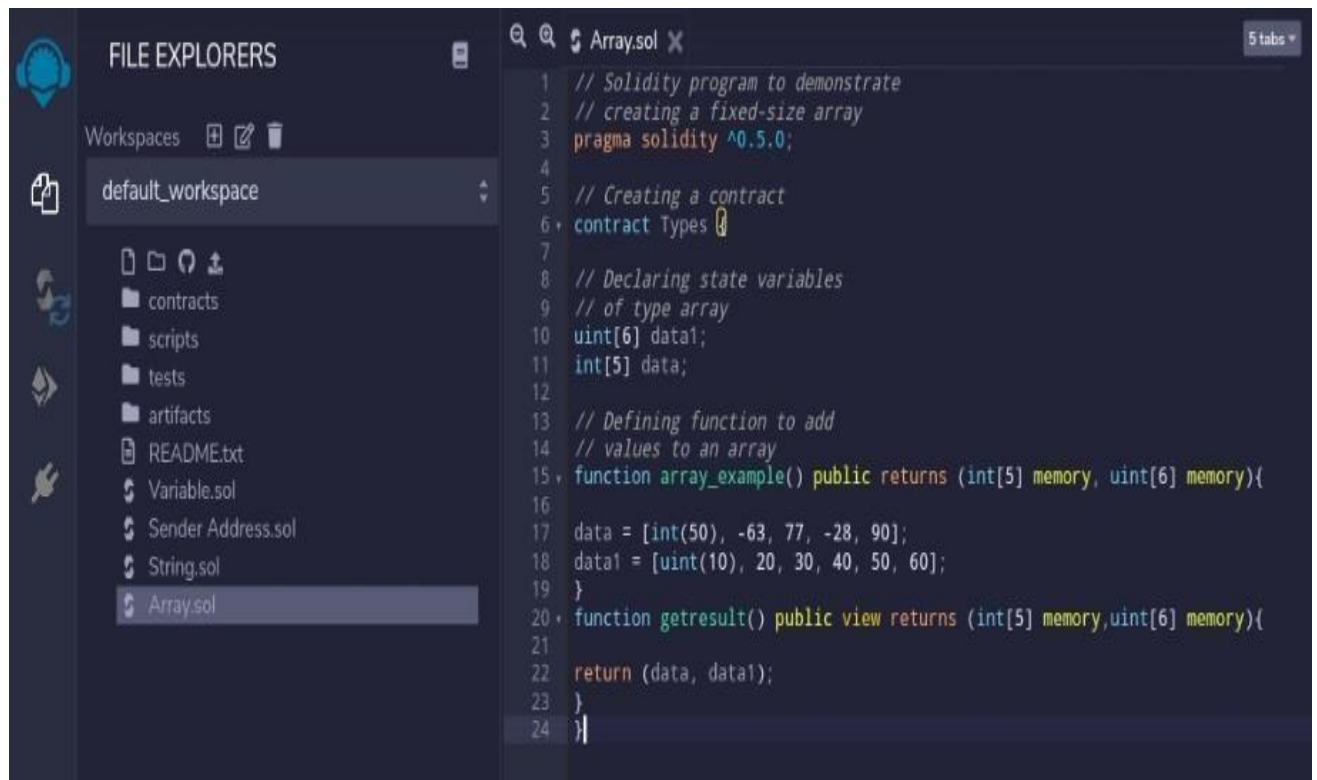
```
// Solidity program to demonstrate //
creating a fixed-size array
pragma solidity ^0.5.0;

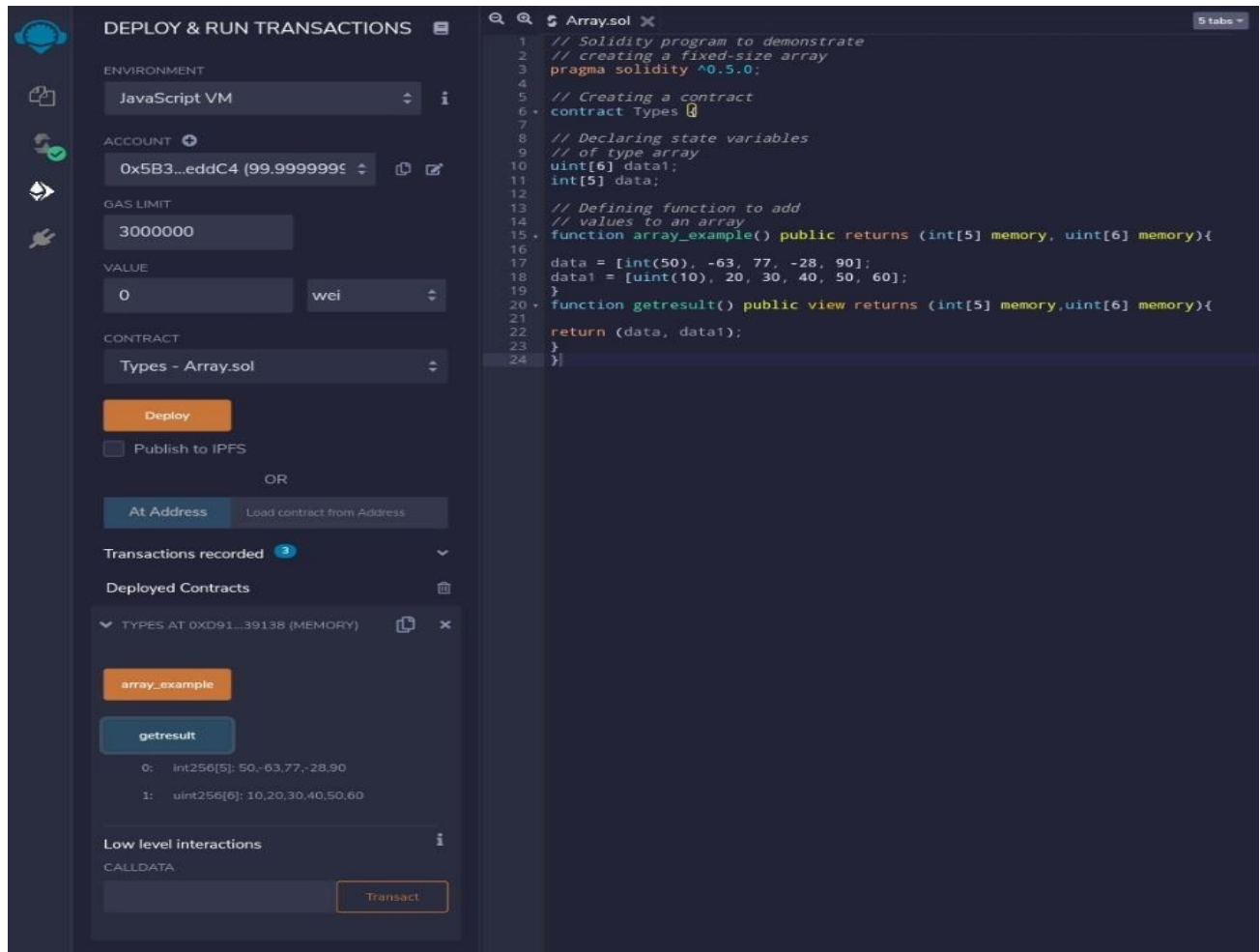
// Creating a contract
contract Types {

// Declaring state variables
// of type array
uint[6] data1; int[5]
data;

// Defining function to add //
values to an array
function array_example() public returns (int[5] memory, uint[6] memory){
data = [int(50), -63, 77, -28, 90]; data1 = [uint(10), 20, 30, 40, 50, 60];
}
function getResult() public view returns (int[5] memory,uint[6] memory){ return
(data, data1);
}
}
```

Output:





c.Structs pragma solidity

^0.5.0;

```
contract test {
  struct Book {
    string title; string
    author; uint
    book_id;
  }
  Book book;
  function setBook() public { book
    = Book('Learn Java', 'TP', 1);
  }
  function getBookId() public view returns (uint) { return
    book.book_id;
  }
}
```

Output:

The screenshot displays the Solidity Compiler interface, which is divided into two main sections: "SOLIDITY COMPILER" and "DEPLOY & RUN TRANSACTIONS".

SOLIDITY COMPILER Section:

- COMPILER:** Set to "0.5.17+commit.d19bba13".
- LANGUAGE:** Set to "Solidity".
- EVM VERSION:** Set to "compiler default".
- COMPILER CONFIGURATION:** Includes checkboxes for "Auto compile", "Enable optimization" (set to 200), and "Hide warnings". A "Compile books.sol" button is present.
- CONTRACT:** Set to "test (books.sol)".
- Buttons:** "Publish on Swarm", "Publish on Ipfs", and "Compilation Details".

DEPLOY & RUN TRANSACTIONS Section:

- TEST AT 0x907...B5E99 (MEMORY):** Includes a "str_out" button.
- TEST AT 0xD2A...FD005 (MEM):** Includes a "str_out" button.
- TEST AT 0xDDA...5482D (MEMORY):** Includes a "str_out" button.
- TEST AT 0X0FC...9A836 (MEMORY):** Includes buttons for "setBook" and "getBookId". The "getBookId" button shows a return value of "uint256: 1".
- Low level interactions:** Includes a "CALLDATA" field and a "Transact" button.

Code Editor:

```
1 pragma solidity ^0.5.0;
2 contract test {
3   struct Book {
4     string title;
5     string author;
6     uint book_id;
7   }
8   Book book;
9   function setBook() public {
10    book = Book('Learn Java', 'JP', 1);
11  }
12  function getBookId() public view returns (uint) {
13    return book.book_id;
14  }
15 }
16
```

Transaction Details:

- from:** 0x58380d6a701c568545dcfc803fc8075f95bedd4
- to:** SolidityTest.(constructor)
- gas:** 3000000 gas

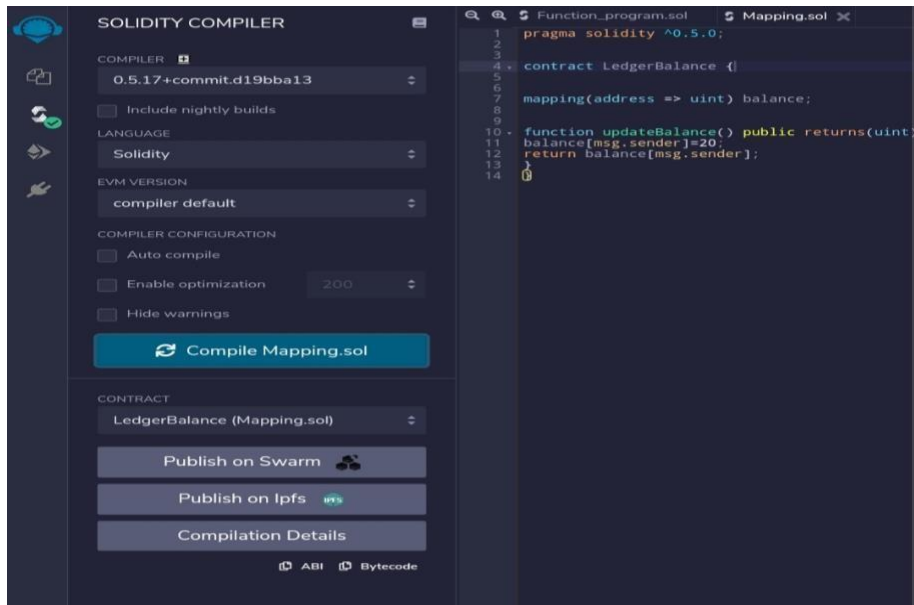
Transaction Cost:

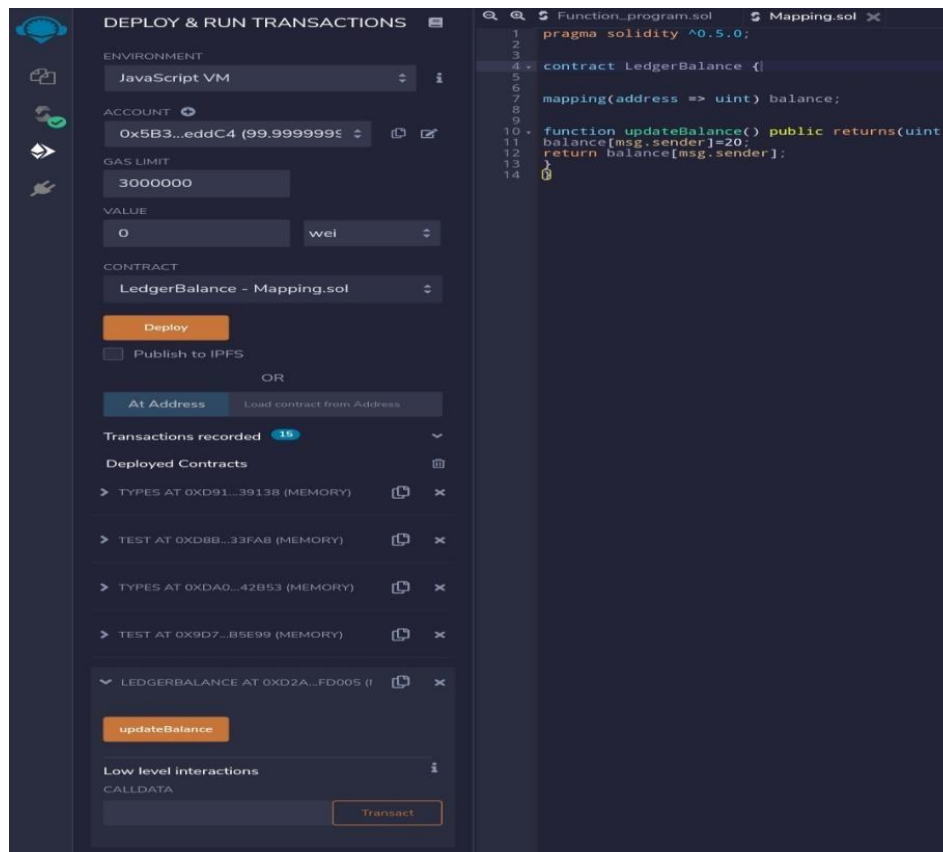
- gas:** 3000000 gas
- transaction cost:** 116859 gas

Mappings

```
pragma solidity ^0.5.0; contract  
LedgerBalance { mapping(address => uint)  
balance; function updateBalance() public  
returns(uint) { balance[msg.sender]=20; return  
balance[msg.sender];  
}  
}
```

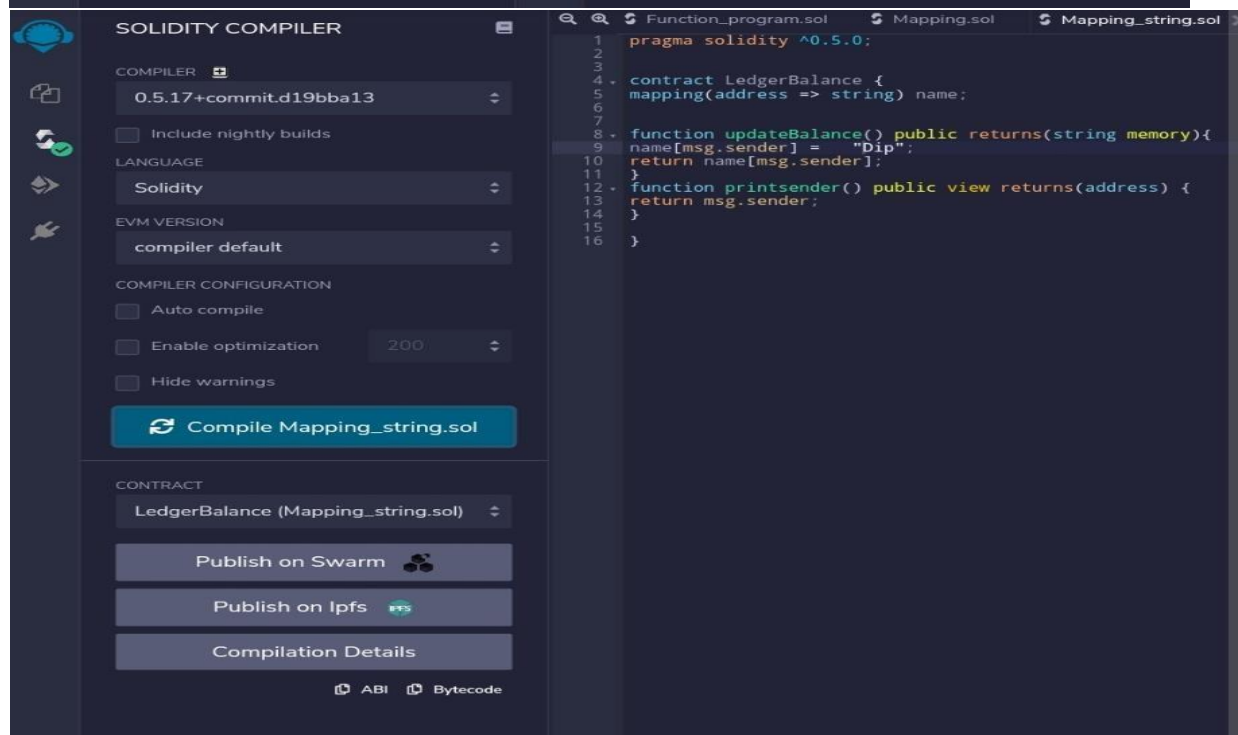
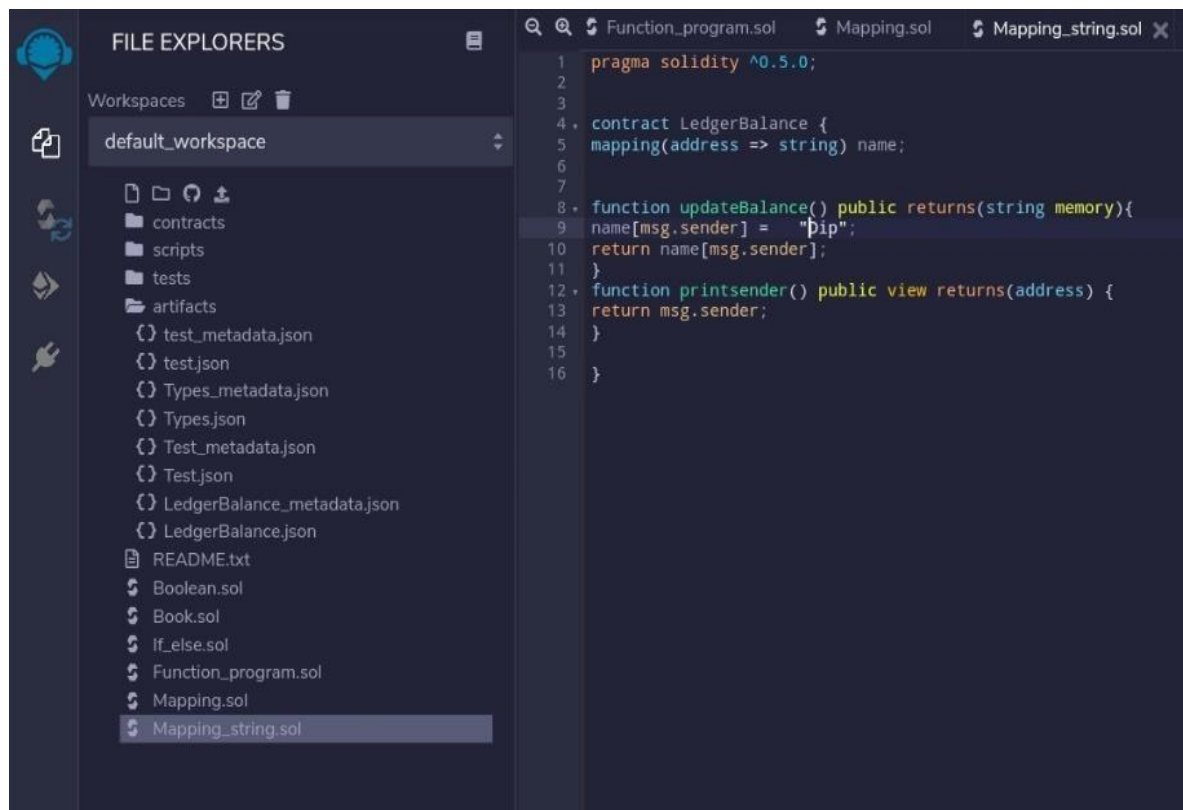
Output:

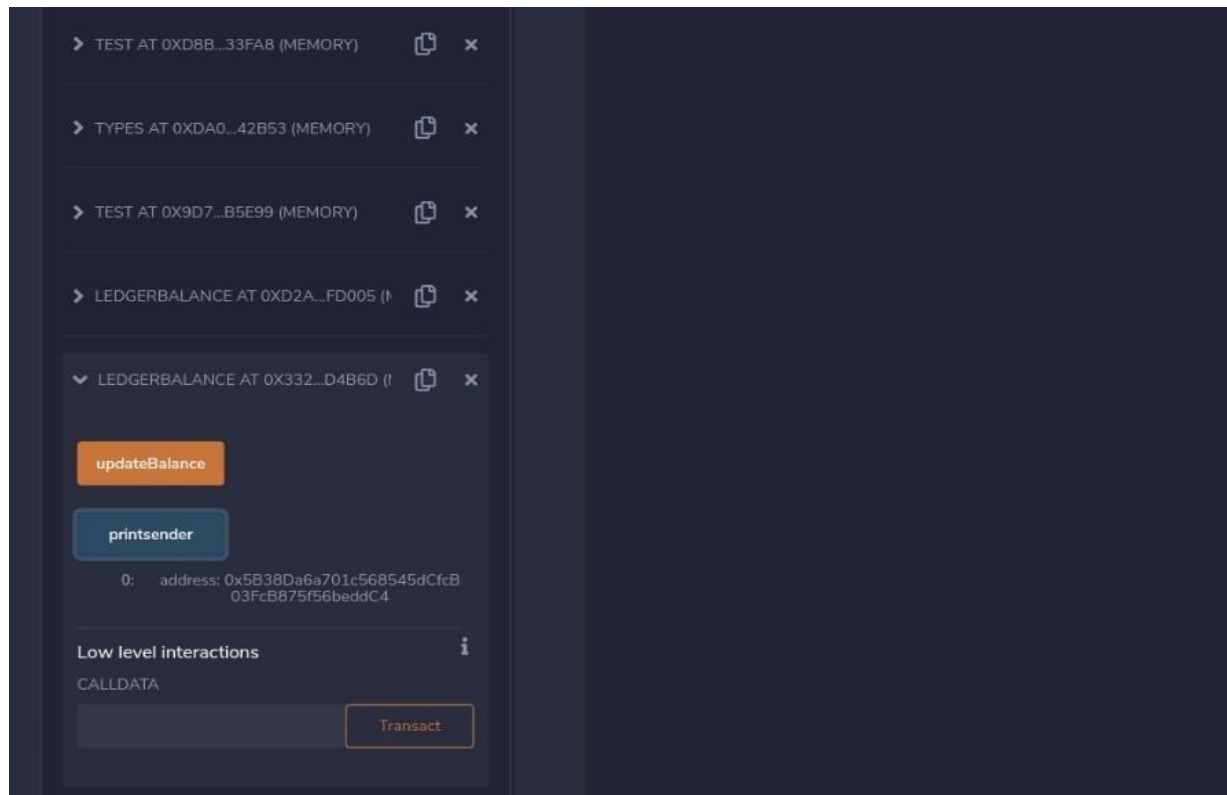




```
pragma solidity ^0.5.0; contract
LedgerBalance {
    mapping(address => string) name;
    function updateBalance() public returns(string memory){
        name[msg.sender] = "Dip"; return name[msg.sender];
    }
    function printsender() public view returns(address) { return
    msg.sender;
    }
}
```

Output:





Practical 6

Implement and demonstrate the use of the following in Solidity:

- a)** Functions
- b)** View Functions
- c)** Pure Functions
- d)** Fallback Functions
- e)** Function Overloading
- f)** Mathematical Functions
- g)** Cryptographic Functions

A.Functions

```
pragma solidity ^0.5.0; contract
SolidityTest {
function testpgmresult() public view returns(uint){
uint a = 1000; // local variable uint b = 2000; uint
result = a + b;
return result; //access the state variable
}
}
```

Output:



SOLIDITY COMPILER

COMPILER 

0.5.17+commit.d19bba13 

☐ Include nightly builds

LANGUAGE

Solidity 

EVM VERSION

compiler default 

COMPILER CONFIGURATION

☐ Auto compile

☐ Enable optimization 200 

☐ Hide warnings

Compile Function test.sol

CONTRACT

SolidityTest (Function test.sol) 

Publish on Swarm 

Publish on Ipfs 

Compilation Details

ABI  Bytecode 

Home

Function test.sol

Functiongetresult.sol

```
1 pragma solidity ^0.5.0;
2 contract SolidityTest {
3
4     function testpgmresult() public view returns(uint){
5         uint a = 1000; // local variable
6         uint b = 2000;
7         uint result = a + b;
8         return result; //access the state variable
9     }
10 }
```



DEPLOY & RUN TRANSACTIONS

ENVIRONMENT

JavaScript VM 

ACCOUNT 

0x5B3...eddC4 (99.9999999€   

GAS LIMIT

3000000

VALUE

0 

wei 

CONTRACT

SolidityTest - Function test.sol 

Deploy

☐ Publish to IPFS

OR

At Address 

Load contract from Address

Transactions recorded 2 

Deployed Contracts 

SOLIDITYTEST AT 0XD91...39138 (MEM  

SOLIDITYTEST AT 0XD8B...33FA8 (MEM  

testpgmresult

0: uint256: 3000

Low level interactions 

CALLDATA

Transact

Home

Function test.sol

Functiongetresult.sol

```
1 pragma solidity ^0.5.0;
2 contract SolidityTest {
3
4     function testpgmresult() public view returns(uint){
5         uint a = 1000; // local variable
6         uint b = 2000;
7         uint result = a + b;
8         return result; //access the state variable
9     }
10 }
```

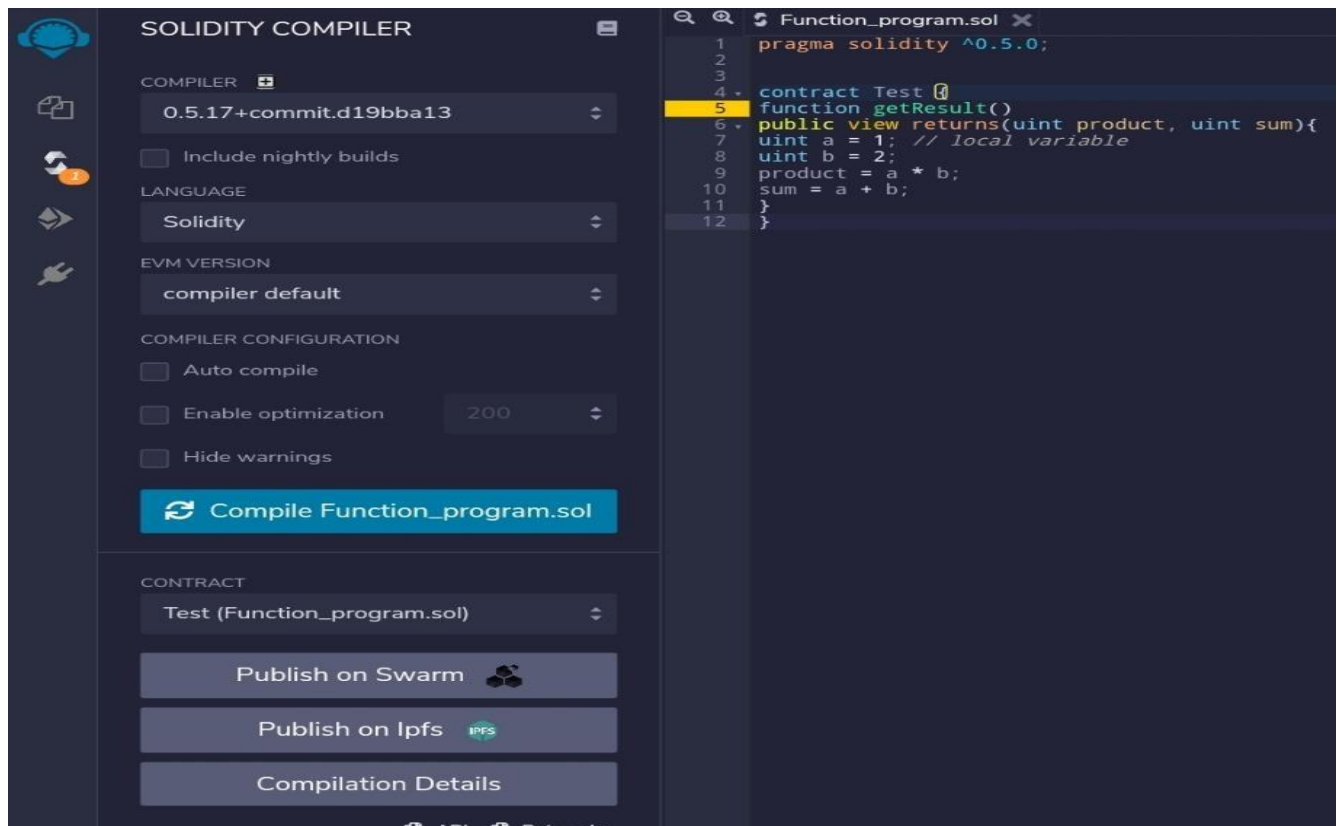
B.View Functions pragma

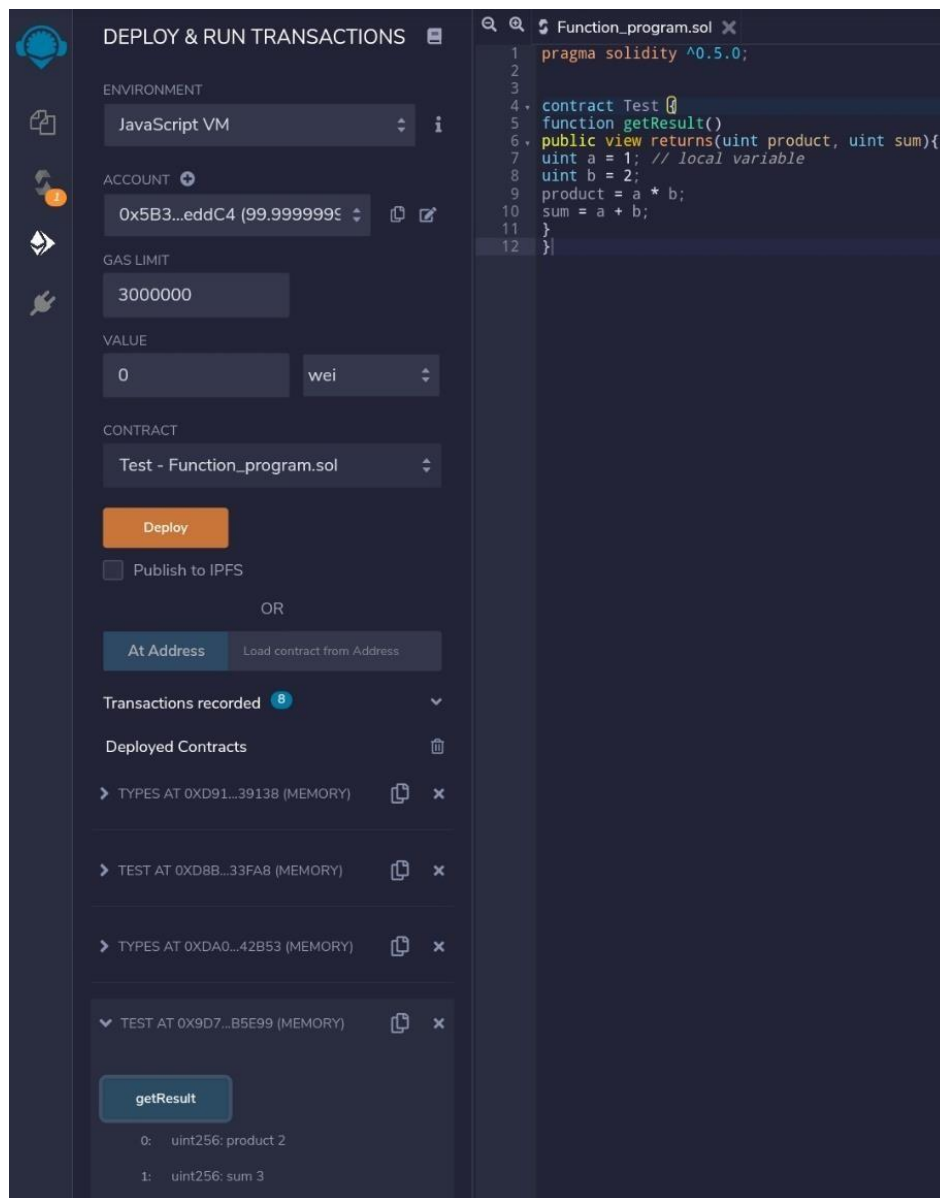
```
solidity ^0.5.0;

contract Test {

function getResult() public view returns(uint product, uint sum){
uint a = 1; // local variable uint b = 2; product = a * b; sum = a +
b;
}
}
```

Output:





C. Pure Functions pragma

solidity ^0.5.0;

contract C { //private

state variable uint

private data;

//public state variable

uint public info;

//constructor constructor()

public { info = 10;

```

}
//private function
function increment(uint a) private pure returns(uint) { return a + 1; }

//public function
function updateData(uint a) public { data = a; } function
getData() public view returns(uint) { return data; }
function compute(uint a, uint b) internal pure returns (uint) { return a + b; } }

//Derived Contract
contract E is C {
uint private result; C
private c;

constructor() public { c
= new C();
}
function getComputedResult() public {
result = compute(3, 5);
} function getResult() public view returns(uint) { return result;
} function getData() public view returns(uint) { return c.info();
} }

```

Output:

SOLIDITY COMPILER

COMPILER

0.5.17+commitd19bba13

Include nightly builds

LANGUAGE

Solidity

EVM VERSION

compiler default

COMPILER CONFIGURATION

Auto compile

Enable optimization200

Hide warnings

Compile calling function external.sol

CONTRACT

C (calling function external.sol)

Publish on Swarm

Publish on Ipfs

Compilation Details

1

pragma solidity ^0.5.0;

2

contract C {

3

//private state variable

4

uint private data;

5

6

//public state variable

7

uint public info;

8

9

//constructor

10

constructor() public {

11

info = 10;

12

}

13

//private function

14

function increment(uint a) private pure returns(uint) { return a + 1; }

15

16

//public function

17

function updateData(uint a) public { data = a; }

18

function getData() public view returns(uint) { return data; }

19

function compute(uint a, uint b) internal pure returns (uint) { return a + b; }

20

}

21

22

//Derived Contract

23

contract E is C {

24

uint private result;

25

C private c;

26

27

constructor() public {

28

c = new C();

29

}

30

function getComputedResult() public {

31

result = compute(3, 5);

32

}

33

function getResult() public view returns(uint) { return result; }

34

function getData() public view returns(uint) { return c.info(); }

35

}

36

listen on network

Search with transaction hash or address

from

0x58380da6701c568545dcfc803fc8875f96bed6c4

to

SolidityTest.(constructor)

gas

3000000 gas

DEPLOY & RUN TRANSACTIONS

TEST AT 0X0FC...9A836 (MEMORY)

setBook

getBookId

0: uint256 1

Low level interactions

CALLDATA

Transact

C AT 0XAED...968BB (MEMORY)

updateData

25

transact

getData

0: uint256 25

info

0: uint256 10

1

pragma solidity ^0.5.0;

2

contract C {

3

//private state variable

4

uint private data;

5

6

//public state variable

7

uint public info;

8

9

//constructor

10

constructor() public {

11

info = 10;

12

}

13

//private function

14

function increment(uint a) private pure returns(uint) { return a + 1; }

15

16

//public function

17

function updateData(uint a) public { data = a; }

18

function getData() public view returns(uint) { return data; }

19

function compute(uint a, uint b) internal pure returns (uint) { return a + b; }

20

}

21

22

//Derived Contract

23

contract E is C {

24

uint private result;

25

C private c;

26

27

constructor() public {

28

c = new C();

29

}

30

function getComputedResult() public {

31

result = compute(3, 5);

32

}

33

function getResult() public view returns(uint) { return result; }

34

function getData() public view returns(uint) { return c.info(); }

35

}

36

listen on network

Search with transaction hash or address

from

0x58380da6701c568545dcfc803fc8875f96bed6c4

to

SolidityTest.(constructor)

gas

3000000 gas

D.Function Overloading

```
pragma solidity ^0.5.0;

contract Test {

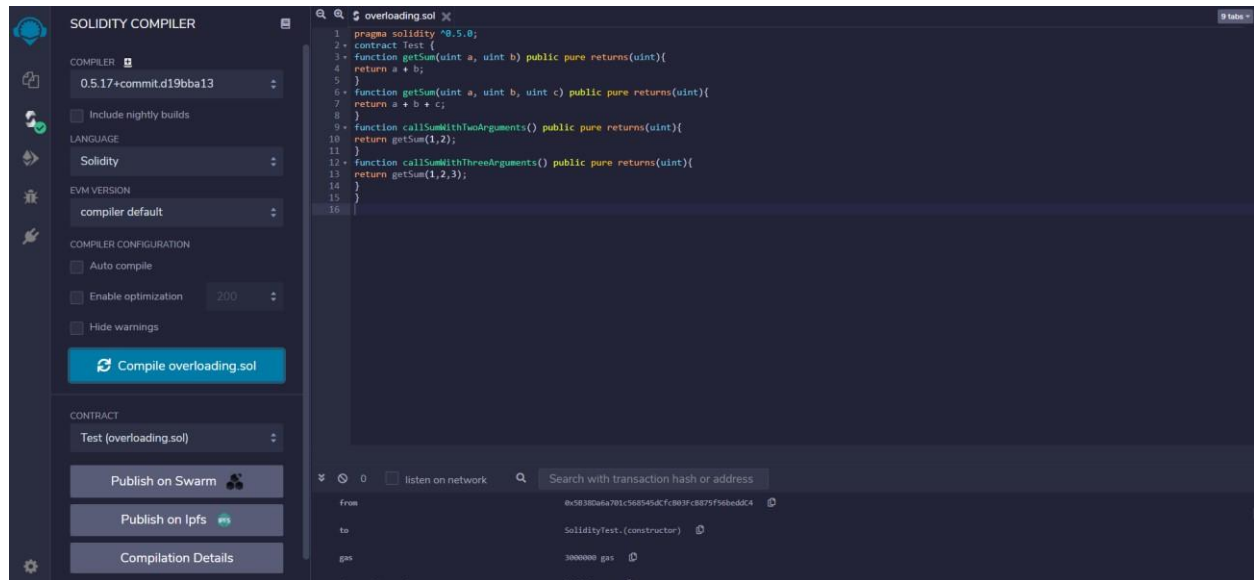
function getSum(uint a, uint b) public pure returns(uint){
return a + b;
}

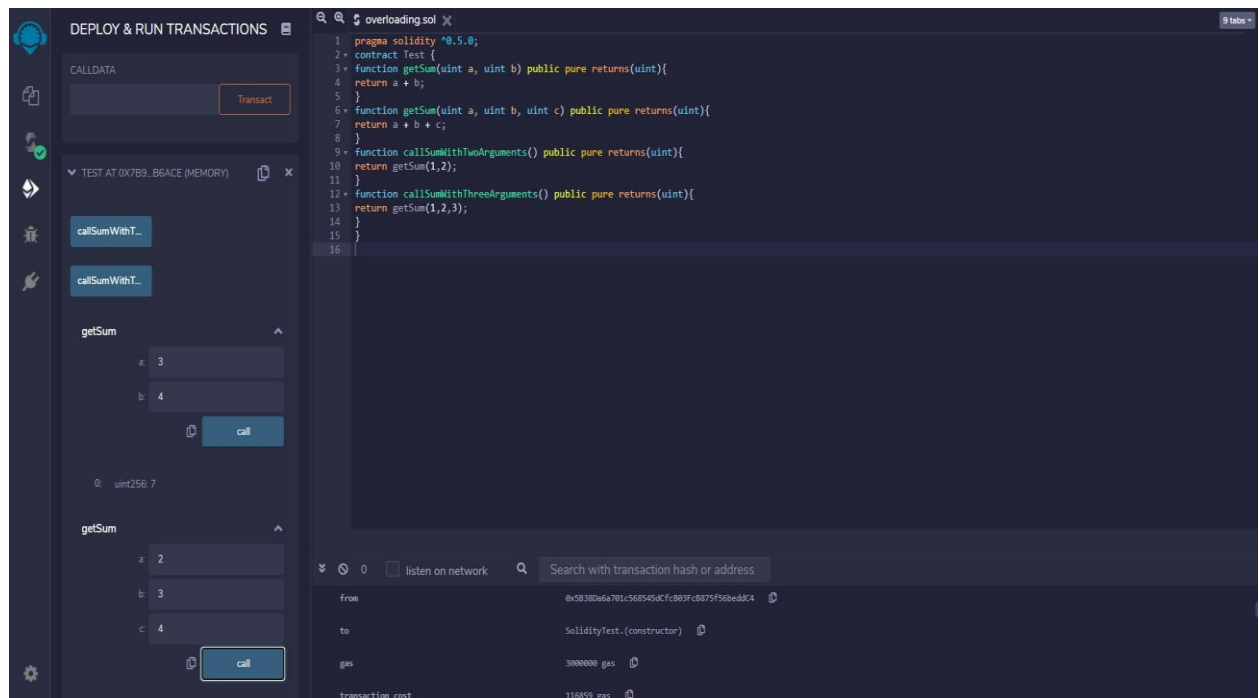
function getSum(uint a, uint b, uint c) public pure returns(uint){ return
a + b + c;
}

function callSumWithTwoArguments() public pure returns(uint){ return
getSum(1,2);
}

function callSumWithThreeArguments() public pure returns(uint){ return
getSum(1,2,3);
}
}
```

Output:





E.Mathematical Functions

```
pragma solidity ^0.5.0;
```

```
contract Test {
```

```
function callAddMod() public pure returns(uint){ return
```

```
addmod(4, 5, 3);
```

```
}
```

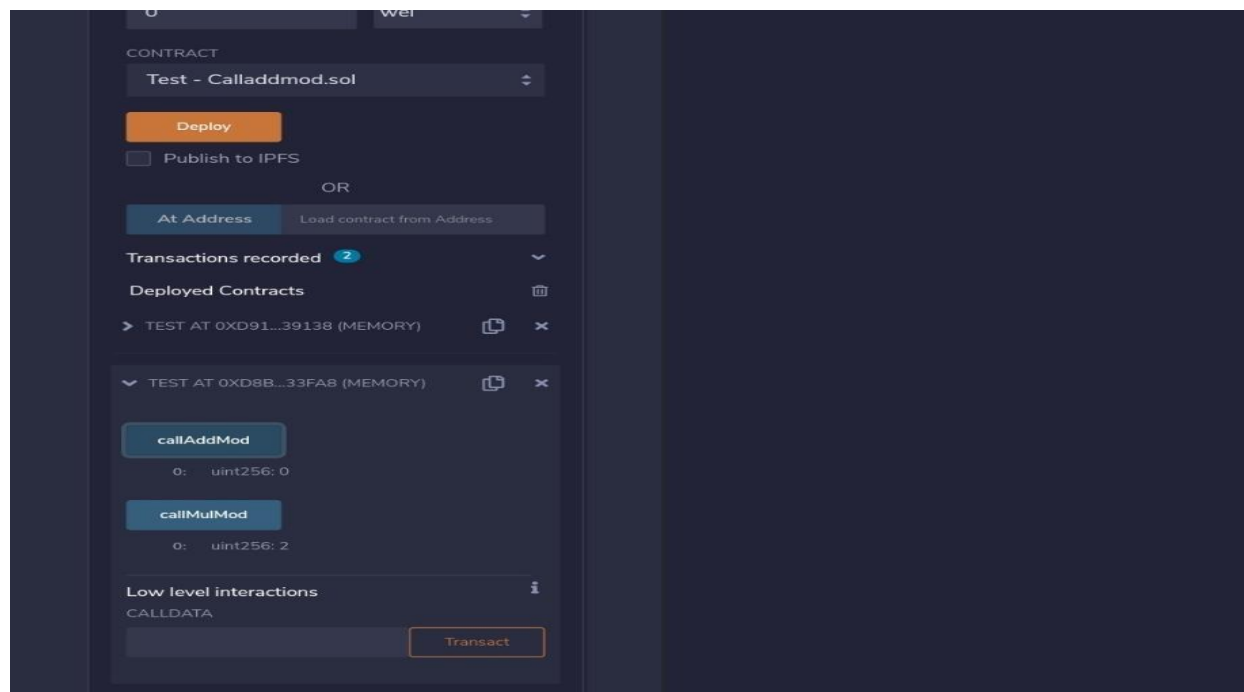
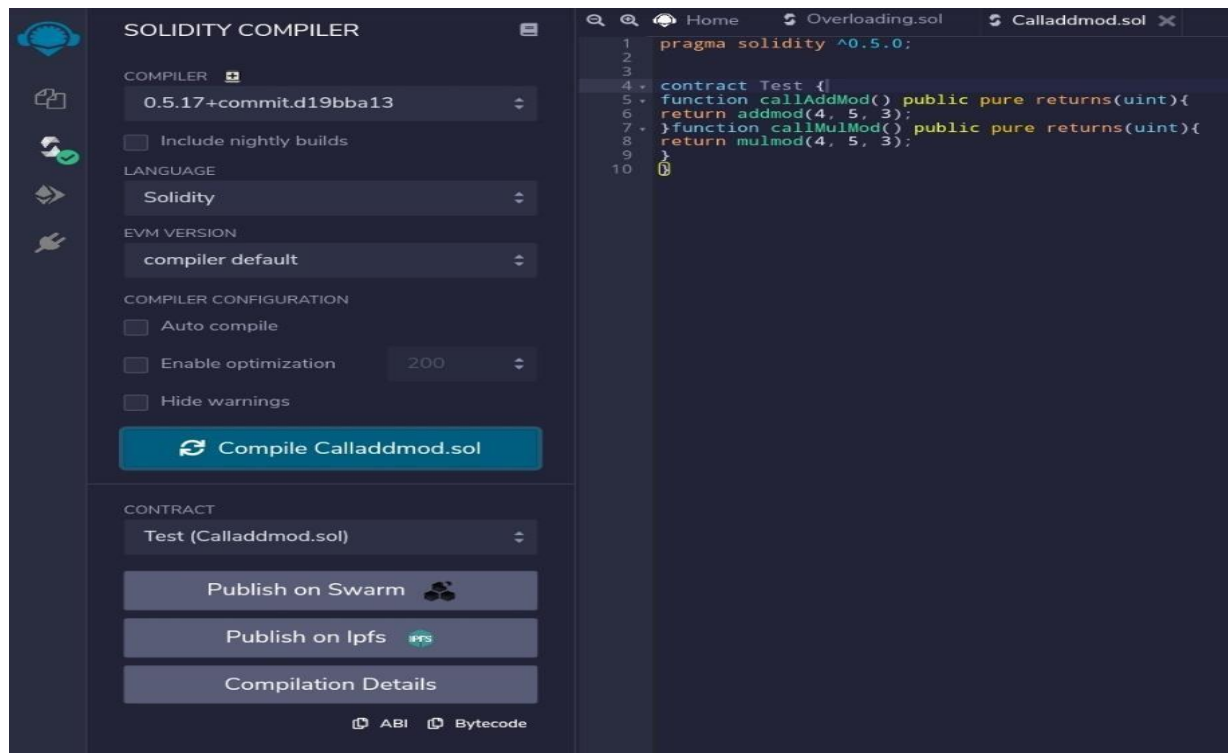
```
function callMulMod() public pure returns(uint){ return
```

```
mulmod(4, 5, 3);
```

```
}
```

```
}
```

Output:



F.Cryptographic Functions pragma
solidity ^0.5.0; contract Test {

```

function callsha256() public pure returns( bytes32 result){
return sha256("ronaldo");
}
function callkeccak256() public pure returns( bytes32 result){ return
keccak256("ronaldo");
}
}

```

Output:

The screenshot displays the Solidity Compiler web interface. On the left, the 'COMPILER' section shows the version '0.5.17+commit.d19bba13' and the language 'Solidity'. The 'CONTRACT' section shows 'Test (cryptodata.sol)' selected. On the right, the code editor shows the Solidity code for the 'Test' contract. The bottom section shows the transaction details for the compilation.

SOLIDITY COMPILER

COMPILER: 0.5.17+commit.d19bba13

LANGUAGE: Solidity

EVM VERSION: compiler default

COMPILER CONFIGURATION:

- ☐ Auto compile
- ☐ Enable optimization (200)
- ☐ Hide warnings

[Compile cryptodata.sol](#)

CONTRACT: Test (cryptodata.sol)

[Publish on Swarm](#)

[Publish on Ipfs](#)

[Compilation Details](#)

overloading.sol cryptodata.sol

```

1 pragma solidity ^0.5.0;
2 contract Test {
3   function callsha256() public pure returns( bytes32 result){
4     return sha256("ronaldo");
5   }
6   function callkeccak256() public pure returns( bytes32 result){
7     return keccak256("ronaldo");
8   }
9 }
10

```

0 ☐ listen on network Search with transaction hash or address

from	to	gas	transaction cost
0x5B38Da6a701c568545dCfcB03Fc8B75f56beddC4	SolidityTest.(constructor)	3000000 gas	116850 gas

DEPLOY & RUN TRANSACTIONS

call

uint256: 9

Low level interactions

CALLDATA

Transact

TEST AT 0x332_D486D (MEMORY)

callkeccak256

bytes32: result 0xdce960f5c0b975b62864be25e4c09cd530f88ff9d23588b6730cbb0e5538eed8

callsha256

bytes32: result 0xe24dd210803b4737a9bd9e3163a4ca807b63201c3bc32b68fb122ca52efff36

Low level interactions

CALLDATA

Transact

overloading.sol

cryptodata.sol

10 tabs

```
1 pragma solidity ^0.5.0;
2 contract Test {
3   function callsha256() public pure returns (bytes32 result){
4     return sha256("ronaldo");
5   }
6   function callkeccak256() public pure returns (bytes32 result){
7     return keccak256("ronaldo");
8   }
9 }
10
```

0

listen on network

Search with transaction hash or address

from	0x58380a6a701c568545dcf883fc8875f56bed6c4
to	SolidityTest.(constructor)
gas	3000000 gas
transaction cost	115000 gas

Practical 7

Implement and demonstrate the use of the following in Solidity:

- a) Contracts
- b) Inheritance
- c) Constructors
- d) Abstract Class
- e) Interfaces

A.Contracts

```
// Solidity program to
// demonstrate how to //
write a smart contract
pragma solidity >= 0.4.16 < 0.7.0;

// Defining a contract contract
Test
{
// Declaring state variables
uint public var1; uint
public var2;
uint public sum;

// Defining public function
// that sets the value of //
the state variable
function set(uint x, uint y) public
{ var1 =
x;
var2=y;
sum=var1+var2;
}

// Defining function to
// print the sum of //
state variables
function get(
) public view returns (uint) { return
sum;
}
}
```

Output:

The screenshot displays the Solidity Compiler interface. On the left, a sidebar contains settings for the compiler (version 0.5.17+commit.d19bba13), language (Solidity), EVM version (compiler default), and compiler configuration (Auto compile, Enable optimization at 200, Hide warnings). Below these are buttons for 'Compile smartcontract.sol', 'Publish on Swarm', 'Publish on Ipfs', and 'Compilation Details'. The main area shows the Solidity code for 'smartcontract.sol', which defines a contract 'Test' with state variables 'var1', 'var2', and 'sum', and functions 'set' and 'get'. The bottom section shows the transaction details for the compilation, including the 'from' address, 'to' address (SolidityTest.(constructor)), and 'gas' used (3000000).

The screenshot displays the Solidity Compiler interface in the 'DEPLOY & RUN TRANSACTIONS' section. The left sidebar shows the 'Low level interactions' panel with a 'Transact' button and a 'TEST AT 0x5E1...4EFF5 (MEMORY)' window. The main area shows the Solidity code for 'smartcontract.sol'. The bottom section shows the transaction details for the deployment, including the 'from' address, 'to' address (Test.set(uint256,uint256)), and 'gas' used (3000000). The transaction status is 'true Transaction mined and execution succeed'.

B.Inheritance


```

// Solidity program to demonstrate Single Inheritance
pragma solidity >=0.4.22 <0.6.0;

// Defining contract
contract parent{

// Declaring internal state variable uint
internal sum;

// Defining external function to set value of internal state variable sum
function setValue() external { uint a = 10; uint b = 20; sum = a + b;
}
}

// Defining child contract
contract child is parent{

// Defining external function to return value of internal state variable sum
function getValue() external view returns(uint) { return sum;
}
}

// Defining calling contract contract
caller {

// Creating child contract object
child cc = new child();

// Defining function to call setValue and getValue functions
function testInheritance() public { cc.setValue();
}
function result() public view returns(uint ){ return
cc.getValue();
}
}

```

Output:

The screenshot displays the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar is active. It shows the environment set to 'JavaScript VM', the account as '0x5B3...eddC4 (99.99999999 wei)', and the gas limit at '3000000'. The contract selected is 'child - Hierarchical.sol'. Below the deployment options, a list of 'Deployed Contracts' is shown, including 'TYPES AT 0XD91...39138 (MEMORY)', 'TEST AT 0XD8B...33FA8 (MEMORY)', 'TYPES AT 0XDA0...42B53 (MEMORY)', 'TEST AT 0X9D7...B5E99 (MEMORY)', 'LEDGERBALANCE AT 0XD2A...FD005 (MEMORY)', 'LEDGERBALANCE AT 0X332...D4B6D (MEMORY)', and 'CHILD AT 0X406...2CFBC (MEMORY)'. The 'CHILD' contract is expanded, showing 'setValue' and 'getValue' buttons. The 'getValue' button is clicked, displaying the output '0: uint256: 30'. At the bottom, the 'Low level interactions' section shows 'CALLDATA' and a 'Transact' button.

On the right, the Solidity code for 'Hierarchical.sol' is visible:

```
1 // Solidity program to
2 // demonstrate
3 // Single inheritance
4 pragma solidity >=0.4.22 <0.6.0;
5
6
7 // Defining contract
8 contract parent{
9
10
11 // Declaring internal
12 // state variable
13 uint internal sum;
14
15 // Defining external function
16 // to set value of internal
17 // state variable sum
18 function setValue() external {
19     uint a = 10;
20     uint b = 20;
21     sum = a + b;
22 }
23
24
25 // Defining child contract
26 contract child is parent{
27
28 // Defining external function
29 // to return value of
30 // internal state variable sum
31 function getValue() external view returns(uint) {
32     return sum;
33 }
34 }
35 }
```

C. Constructors

```
// Solidity program to demonstrate  
// creating a constructor pragma  
solidity ^0.5.0;
```

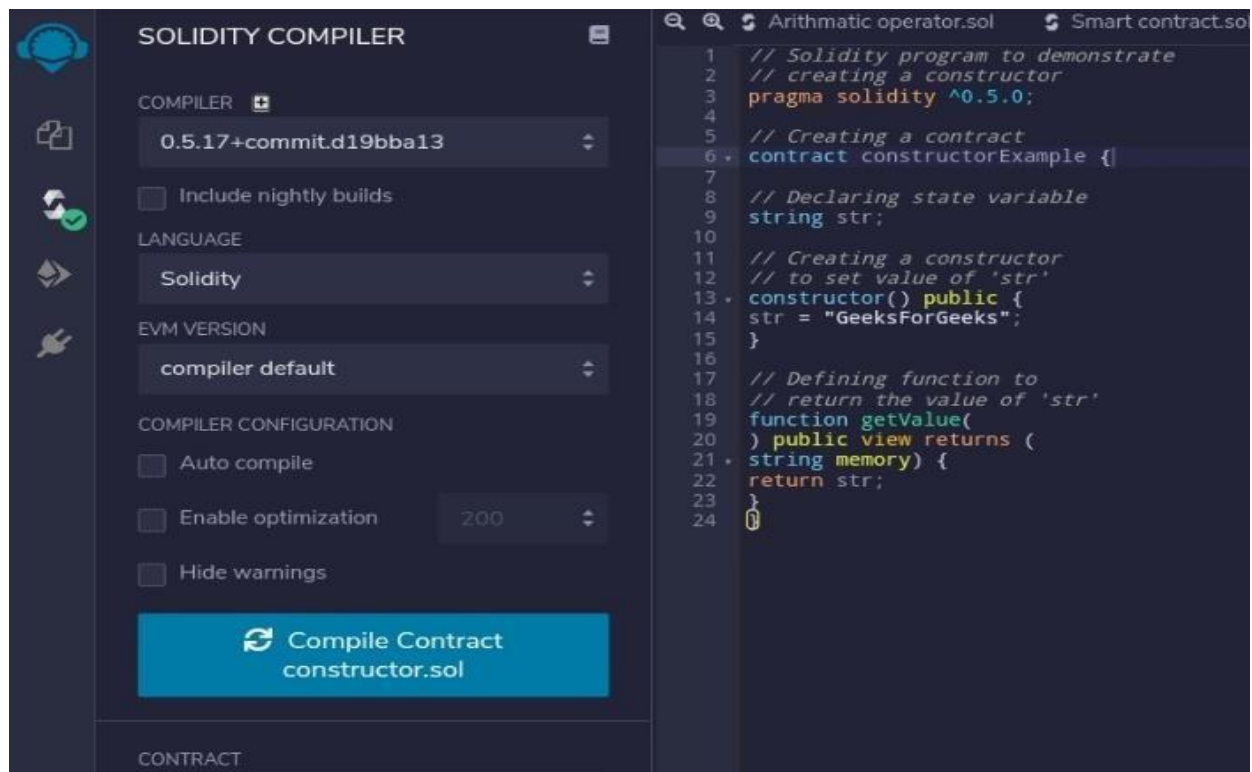
```
// Creating a contract contract  
constructorExample {
```

```
// Declaring state variable string  
str;
```

```
// Creating a constructor  
// to set value of 'str'  
constructor() public { str  
= "GeeksForGeeks";  
}
```

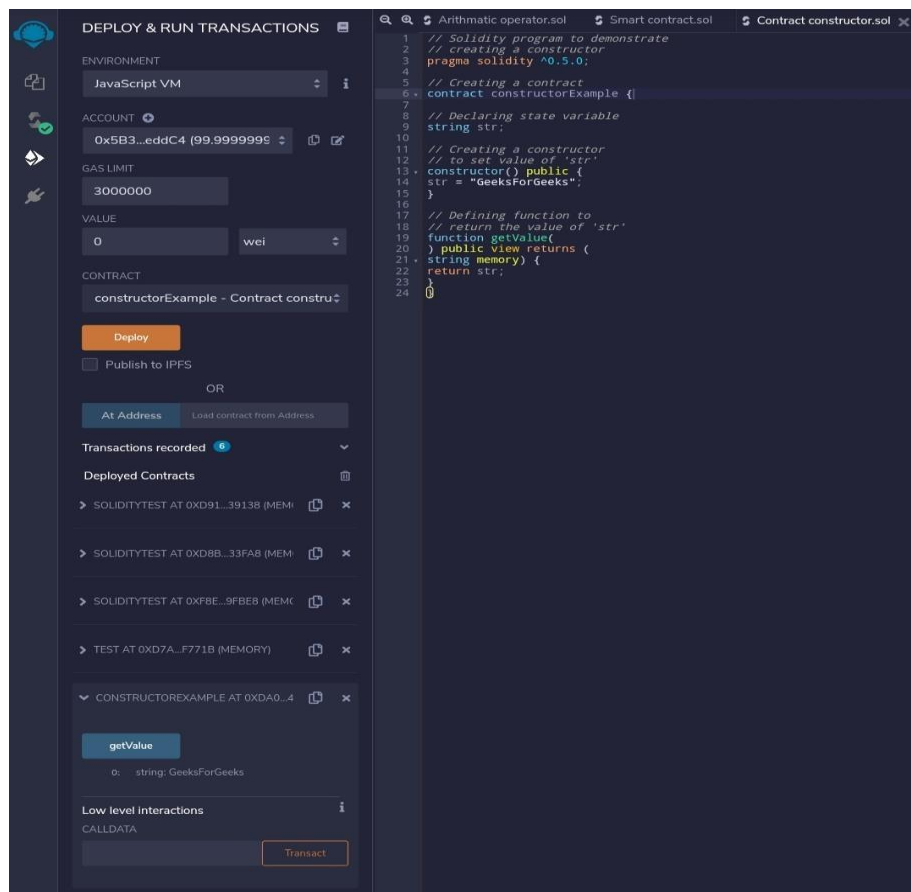
```
// Defining function to //  
return the value of 'str'  
function getValue( )  
public view returns (  
string memory) { return  
str; }  
}
```

Output:



The screenshot displays the Solidity Compiler interface. On the left, the 'COMPILER' section shows the version '0.5.17+commit.d19bba13', with options for 'Include nightly builds', 'LANGUAGE' set to 'Solidity', 'EVM VERSION' set to 'compiler default', and 'COMPILER CONFIGURATION' with 'Auto compile', 'Enable optimization' (set to 200), and 'Hide warnings' all unchecked. A large blue button labeled 'Compile Contract constructor.sol' is visible. The right pane shows the source code for 'constructorExample.sol'.

```
1 // Solidity program to demonstrate
2 // creating a constructor
3 pragma solidity ^0.5.0;
4
5 // Creating a contract
6 contract constructorExample {
7
8     // Declaring state variable
9     string str;
10
11     // Creating a constructor
12     // to set value of 'str'
13     constructor() public {
14         str = "GeeksForGeeks";
15     }
16
17     // Defining function to
18     // return the value of 'str'
19     function getValue(
20         ) public view returns (
21         string memory) {
22         return str;
23     }
24 }
```



The screenshot shows the 'DEPLOY & RUN TRANSACTIONS' interface. The 'ENVIRONMENT' is 'JavaScript VM', 'ACCOUNT' is '0x5B3...eddC4 (99.9999999)', 'GAS LIMIT' is '3000000', 'VALUE' is '0 wei', and the 'CONTRACT' is 'constructorExample - Contract constru...'. A 'Deploy' button is present. Below, a list of 'Deployed Contracts' includes 'SOLIDITYTEST AT 0XD91...3913B (MEM)', 'SOLIDITYTEST AT 0XD8B...33FAB (MEM)', 'SOLIDITYTEST AT 0XFBE...9FBE8 (MEM)', 'TEST AT 0XD7A...F771B (MEMORY)', and 'CONSTRUCTOREXAMPLE AT 0XDA0...4'. The 'CONSTRUCTOREXAMPLE' contract is selected, showing its 'getValue' function with the result '0: string: GeeksForGeeks'. A 'Transact' button is at the bottom.

Practical 8

Implement and demonstrate the use of the following in Solidity:

- a)** Libraries
- b)** Assembly
- c)** Events
- d)** Error Handling

a) Libraries

// Solidity program to demonstrate require statement

```
pragma solidity ^0.5.0;
```

```
// Creating a contract contract
```

```
requireStatement {
```

```
// Defining function to check input
```

```
function checkInput( uint _input)
```

```
public view returns( string
```

```
memory){
```

```
require(_input >= 0, "invalid uint8");
```

```
require(_input <= 255, "invalid uint8"); return
```

```
"Input is Uint8";
```

```
}
```

```
// Defining function to use require statement
```

```
function Odd(uint _input) public view returns(bool){
```

```
require(_input % 2 != 0); return true;
```

```
}
```

```
}
```

Output:

The screenshot shows the Solidity Compiler interface. On the left, the 'COMPILER' section is set to version 0.5.17+commit.d19bba13, with 'Solidity' as the language and 'compiler default' as the EVM version. The 'COMPILER CONFIGURATION' section has 'Auto compile' checked, 'Enable optimization' set to 200, and 'Hide warnings' unchecked. A 'Compile requirestatement.sol' button is visible. Below it, a message states 'No Contract Compiled Yet'. On the right, the source code for 'requirestatement.sol' is displayed, showing a contract named 'requireStatement' with two functions: 'checkInput' and 'Odd'.

```
1 // Solidity program to
2 // demonstrate require
3 // statement
4 pragma solidity ^0.5.0;
5
6
7 // Creating a contract
8 contract requireStatement {
9
10 // Defining function to
11 // check input
12 function checkInput(
13 uint _input) public view returns(
14 string memory){
15 require(_input >= 0, "invalid uint8");
16 require(_input <= 255, "invalid uint8");
17
18 return "Input is Uint8";
19 }
20 // Defining function to
21 // use require statement
22 function Odd(uint _input) public view returns(bool){
23 require(_input % 2 != 0);
24 return true;
25 }
26 }
```

The screenshot shows the 'DEPLOY & RUN TRANSACTIONS' interface. The 'ENVIRONMENT' is set to 'JavaScript VM'. The 'ACCOUNT' is '0x5B3...eddC4 (99.9999999€)'. The 'GAS LIMIT' is '3000000'. The 'VALUE' is '0 wei'. The 'CONTRACT' is 'requireStatement - requirestatement.sc'. The 'Deploy' button is highlighted. Below it, the 'Deployed Contracts' section shows the contract 'REQUIRESTATEMENT AT 0XD91...3913I'. The 'checkInput' function is called with input '254', returning '0: string: Input is Uint8'. The 'Odd' function is called with input '253', returning '0: bool: true'. The 'Low level interactions' section shows the 'CALLDATA' field and a 'Transact' button.

b) Assembly

// Solidity program to demonstrate assert statement

```
pragma solidity ^0.5.0;
```

```
// Creating a contract contract
```

```
assertStatement {
```

```
// Defining a state variable bool
```

```
result;
```

```
// Defining a function to check condition
```

```
function checkOverflow( uint _num1,
```

```
uint _num2) public { uint8 sum =
```

```
_num1 + _num2; assert(sum<=255);
```

```
result = true;
```

```
}
```

```
// Defining a function to print result of assert statement
```

```
function getResult() public view returns(string memory){
```

```
if(result == true){ return "No Overflow";
```

```
} else{ return
```

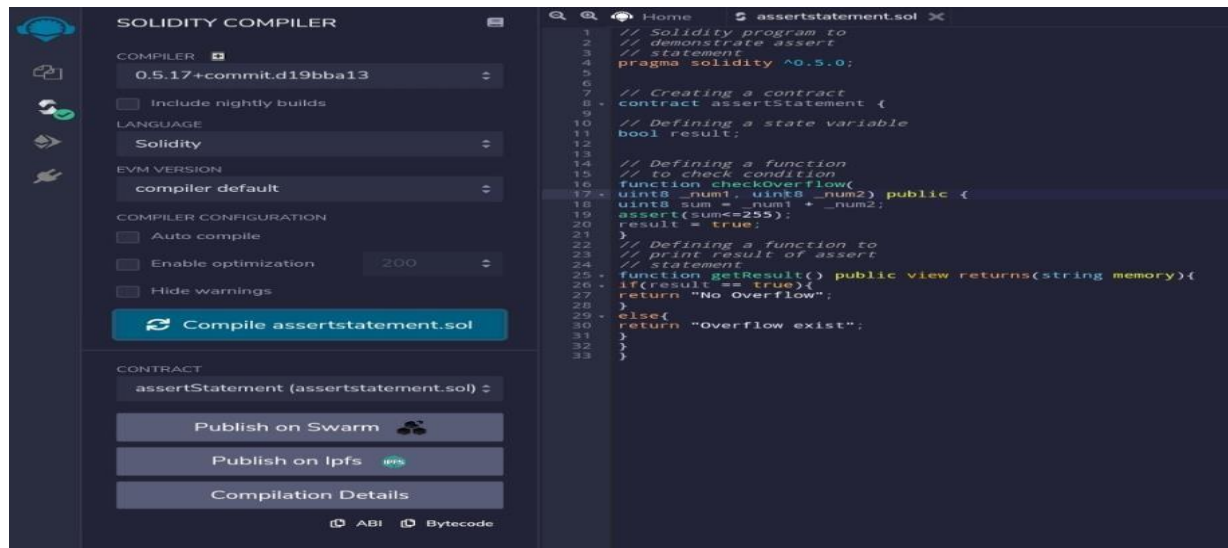
```
"Overflow exist";
```

```
}
```

```
}
```

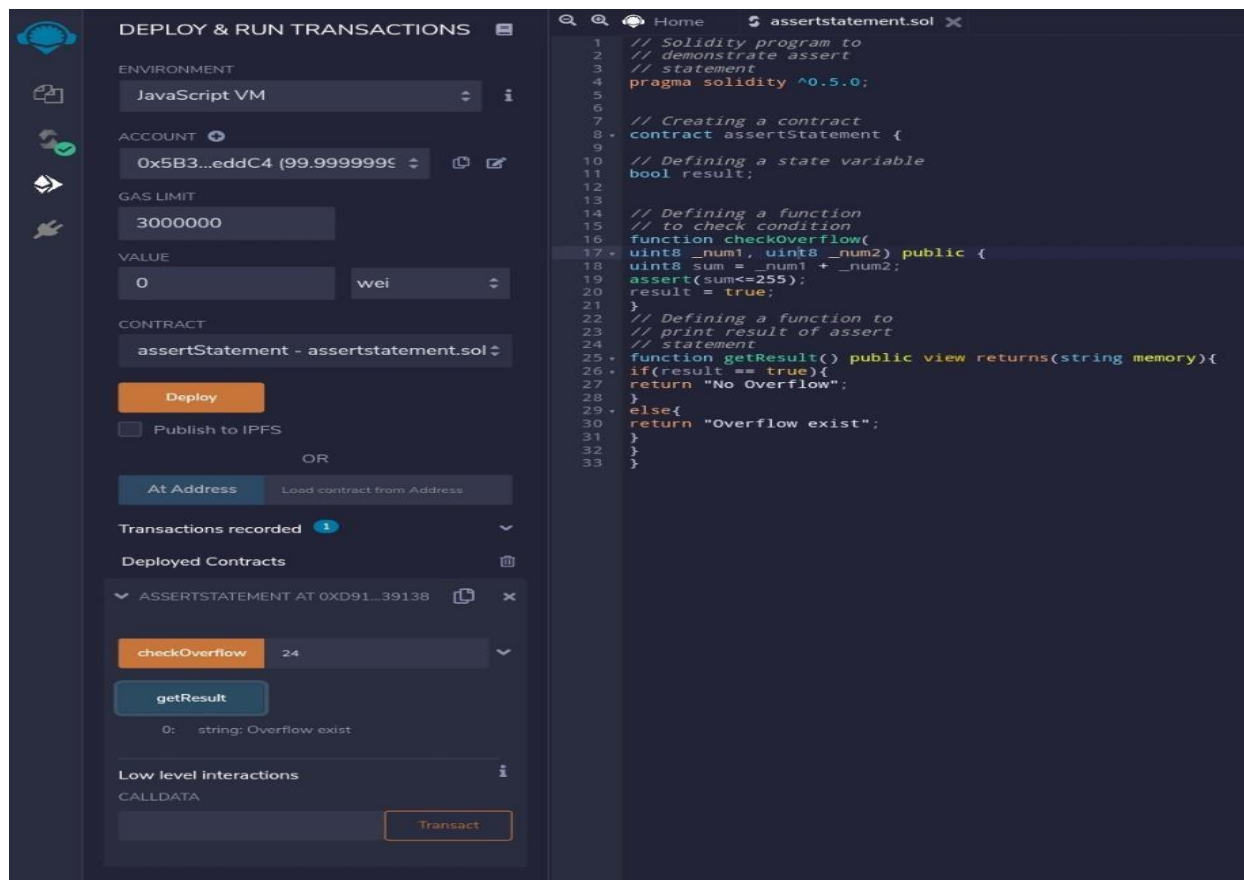
```
}
```

Output:



The screenshot shows the Solidity Compiler interface. On the left, the 'COMPILER' section is set to '0.5.17+commit.d19bba13'. The 'LANGUAGE' is 'Solidity' and the 'EVM VERSION' is 'compiler default'. The 'COMPILER CONFIGURATION' has 'Auto compile' checked, 'Enable optimization' set to '200', and 'Hide warnings' checked. A 'Compile assertstatement.sol' button is visible. Below this, the 'CONTRACT' section shows 'assertStatement (assertstatement.sol)'. On the right, the code editor displays the Solidity code for 'assertstatement.sol'.

```
1 // Solidity program to
2 // demonstrate assert
3 // statement
4 pragma solidity ^0.5.0;
5
6
7 // Creating a contract
8 contract assertStatement {
9
10 // Defining a state variable
11 bool result;
12
13
14 // Defining a function
15 // to check condition
16 function checkOverflow(
17     uint8 _num1, uint8 _num2) public {
18     uint8 sum = _num1 + _num2;
19     assert(sum <= 255);
20     result = true;
21 }
22
23 // Defining a function to
24 // print result of assert
25 // statement
26 function getResult() public view returns(string memory){
27     if(result == true){
28         return "No Overflow";
29     }
30     else{
31         return "Overflow exist";
32     }
33 }
```



The screenshot shows the 'DEPLOY & RUN TRANSACTIONS' interface. The 'ENVIRONMENT' is 'JavaScript VM'. The 'ACCOUNT' is '0x5B3...eddC4 (99.999999%)'. The 'GAS LIMIT' is '3000000'. The 'VALUE' is '0' in 'wei'. The 'CONTRACT' is 'assertStatement - assertstatement.sol'. A 'Deploy' button is visible. Below this, the 'Transactions recorded' section shows '1' transaction. The 'Deployed Contracts' section shows 'ASSERTSTATEMENT AT 0XD91...39138'. The 'checkOverflow' function is selected, and the 'getResult' button is visible. The output shows '0: string: Overflow exist'. The 'Low level interactions' section shows 'CALLDATA' and a 'Transact' button.

c) Events

// Solidity program to demonstrate assert statement

pragma solidity ^0.5.0;

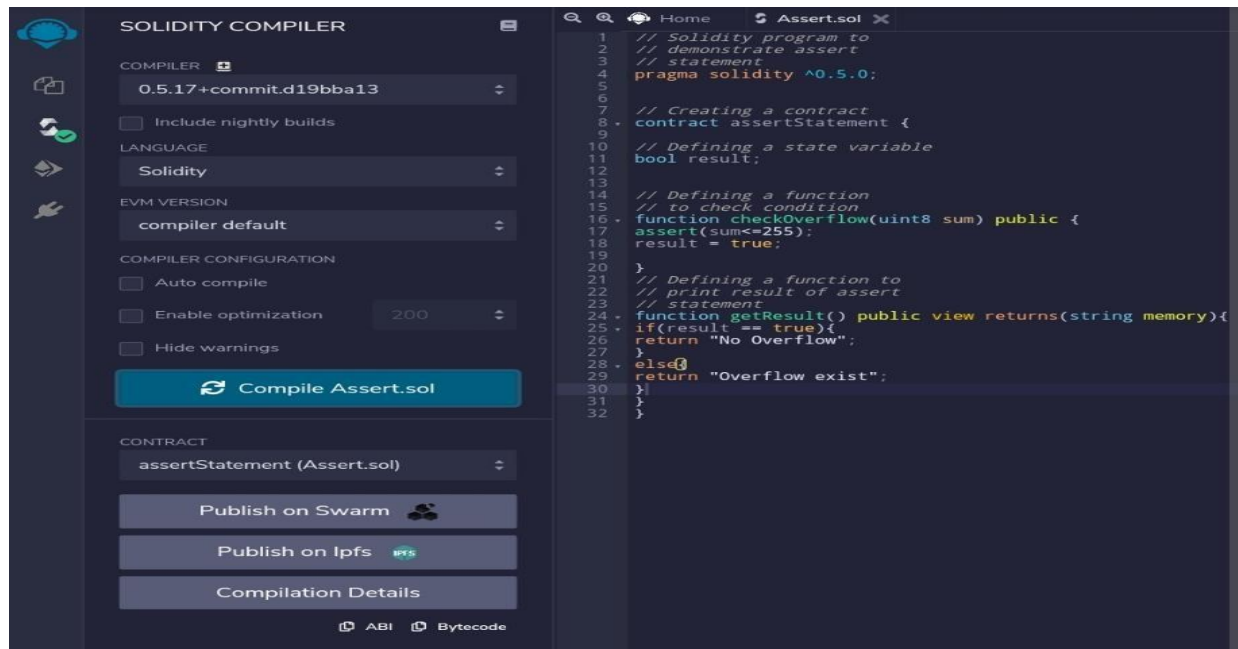

```
// Creating a contract contract
assertStatement {

// Defining a state variable bool
result;

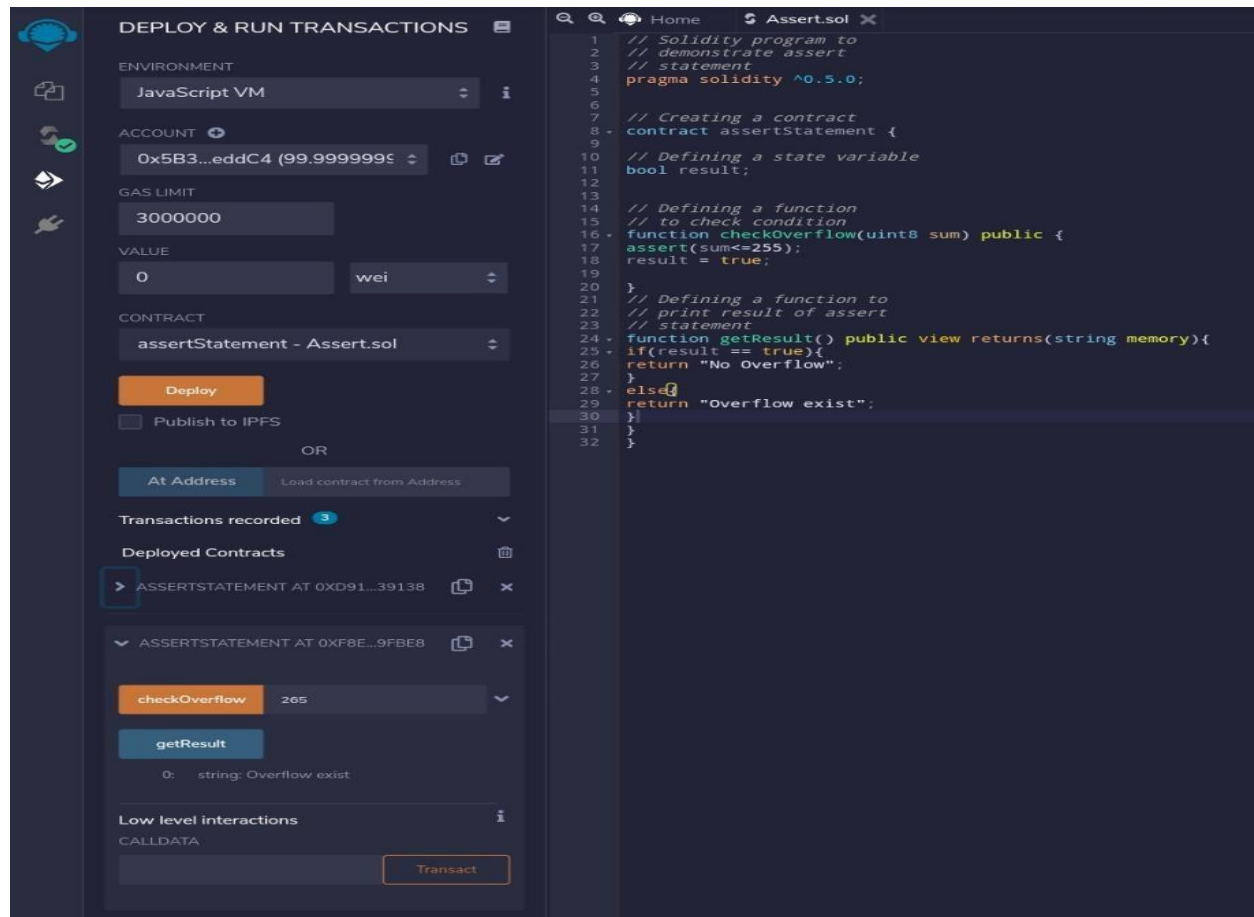
// Defining a function //
to check condition
function checkOverflow(uint8 sum) public {
assert(sum<=255); result = true;
}

// Defining a function to print result of assert statement
function getResult() public view returns(string memory){
if(result == true){ return "No Overflow";
} else{ return
"Overflow exist";
}
}
}
```

Output:



The screenshot shows the Solidity Compiler interface. On the left, the 'COMPILER' section is set to '0.5.17+commit.d19bba13'. The 'LANGUAGE' is 'Solidity' and the 'EVM VERSION' is 'compiler default'. The 'COMPILER CONFIGURATION' section has 'Auto compile' checked, 'Enable optimization' set to '200', and 'Hide warnings' checked. A 'Compile Assert.sol' button is visible. Below this, the 'CONTRACT' section shows 'assertStatement (Assert.sol)' with buttons for 'Publish on Swarm', 'Publish on Ipfs', and 'Compilation Details'. On the right, the code for 'Assert.sol' is displayed, showing a contract named 'assertStatement' with a state variable 'bool result', a function 'checkOverflow' that asserts 'sum <= 255', and a function 'getResult' that returns 'No Overflow' or 'Overflow exist' based on the 'result' variable.



The screenshot shows the 'DEPLOY & RUN TRANSACTIONS' interface. The 'ENVIRONMENT' is 'JavaScript VM'. The 'ACCOUNT' is '0x5B3...eddC4 (99.99999999 wei)'. The 'GAS LIMIT' is '3000000'. The 'VALUE' is '0 wei'. The 'CONTRACT' is 'assertStatement - Assert.sol'. A 'Deploy' button is visible. Below this, the 'Transactions recorded' section shows '3' transactions. The 'Deployed Contracts' section shows two contracts: 'ASSERTSTATEMENT AT 0XD91...39138' and 'ASSERTSTATEMENT AT 0XF8E...9FBE8'. The second contract is expanded, showing a 'checkOverflow' function with a value of '265' and a 'getResult' function that returns 'string: Overflow exist'. The 'Low level interactions' section shows 'CALLDATA' and a 'Transact' button.

d) Error Handling

// Solidity program to demonstrate revert statement

```
pragma solidity ^0.5.0;
```

```
// Creating a contract contract
```

```
revertStatement {
```

```
// Defining a function to check condition
```

```
function checkOverflow( uint _num1, uint  
_num2) public view returns( string memory,
```

```
uint) { uint sum = _num1 + _num2; if(sum <  
0 || sum > 255){ revert(" Overflow Exist");
```

```
} else{
```

```
return ("No Overflow", sum);
```

```
}
```

```
}
```

```
}
```

```
}
```

Output:

The screenshot displays the Solidity Compiler interface. On the left, the 'COMPILER' section shows the version '0.5.17+commit.d19bba13' and the language 'Solidity'. The 'CONTRACT' section shows 'revertStatement (revertstatement.sol)'. The 'COMPILER CONFIGURATION' section has 'Auto compile' checked and 'Enable optimization' set to '200'. A 'Compile revertstatement.sol' button is visible. The main editor shows the Solidity code for 'revertstatement.sol', which includes a function 'checkOverflow' that checks for integer overflow and reverts if it occurs. The bottom panel shows the transaction status: '[vm] from: 0x5B3...eddC4 to: Test.set(uint256,uint256) 0x5e1...4eff5 value: 0 wei data: 0x1ab...0000f logs: 0 hash: 0x85f...143c6'. The transaction hash is '0x85fca5de9819161eb66c32b59aabd8e9c427b740d75c05a17394c2854b3143c6'.

The screenshot displays the Solidity Compiler interface, specifically the 'DEPLOY & RUN TRANSACTIONS' section. The 'var1' and 'var2' fields are set to '52' and '35' respectively. The 'CALLDATA' field is empty. The 'Transact' button is highlighted. The 'REVERTSTATEMENT AT 0x1C9_2B4BC' section shows the 'checkOverflow' function being called with arguments 'uint256: 52' and 'uint256: 35'. The output shows '0: string: No Overflow' and '1: uint256: 87'. The bottom panel shows the transaction status: '[vm] from: 0x5B3...eddC4 to: Test.set(uint256,uint256) 0x5e1...4eff5 value: 0 wei data: 0x1ab...0000f logs: 0 hash: 0x85f...143c6'. The transaction hash is '0x85fca5de9819161eb66c32b59aabd8e9c427b740d75c05a17394c2854b3143c6'.